

Groovy and Grails

Ken Kousen

2019-04-24

Table of Contents

Set Up	1
Groovy	2
Introduction	3
Installing Groovy	4
Hello, World	6
Running Groovy	8
The groovyc and groovy commands	8
The Groovy Shell	9
The Groovy Console	9
Numbers	11
assert, imports, and def	12
assert	12
Automatic imports	12
The def keyword	13
Strings and Groovy Strings	14
Operator Overloading	16
POJOs vs POGOs	17
AST Transformations	22
The @ToString transformation	22
The @EqualsAndHashCode transformation	24
The @TupleConstructor transformation	25
The @Canonical transformation	25
Collections	27
Lists	27
Sets	28
Maps	29
Closures	30
Iterating with loops	30
Processing Lists with Closures	30
The spread-dot operator	33
Maps and closures	33
More on closures (optional)	34
Miscellaneous operators	36
Safe navigation	36
Spaceship	36
The Groovy Truth	36
Elvis	37
Method References	37

Groovy and XML	40
Parsing XML	40
Generating XML	42
Groovy and JSON	43
Parsing JSON data	43
Generating JSON	43
Grails	44
Grails Introduction	45
Installing Grails	45
Grails Links	46
Starting a Grails project	47
Project Structure	47
Controllers	49
Accessing Controllers	49
Returning from Controller Actions	50
The params map	51
Default Action in a Controller	52
Grails Object Relational Mapping (GORM)	53
Logging SQL	53
Optional Hibernate-managed Properties	54
Constraints	55
Saving to the Database	56
The Hibernate Session	58
GORM Relationships	60
One-to-Many	60
Many-to-Many	61
Grails Queries	64
Dynamic Finders	64
Criteria Queries	64
Where Queries	65
Services	67
The need for a service layer	67
Grails and the DAO layer	68
Grails without transactions	69
Grails services	70
REST	71
REST in Grails (Grails 2.2 and earlier)	71
REST in Grails (Grails 2.3 and above)	72
Exercises	73
1. Fundamentals	74
2. Collections	76

3. Groovy Exercises	79
4. Grails Request Processing	80
5. Creating A New Grails Application	85
6. Adding Another Domain Class	87
7. Unit Testing Domain Classes	89
8. Dynamic finders and criteria queries, part 1	92
9. Dynamic finders and criteria queries, part 2	94
10. Complete the domain model.....	97
11. Create a Geocoder service	101
12. Using the Google Visualization plugin	106
13. Switching databases	108
14. Mapping to existing tables	111
15. Calling stored procedures	116

Set Up

These materials are based on the latest Groovy distribution in the 2.4 series and the latest Grails distribution in the 3.3 series.

Groovy

Introduction

Groovy is one of the family of languages that compile to bytecodes that run on the Java virtual machine.

Popular non-Java languages on the JVM

Groovy

Most mature and easiest for Java developers to learn

Scala

Statically typed with immutables; a good transitional language from OOP to functional programming

Clojure

Lisp on the JVM; pure functional language with immutables

Kotlin

Statically typed language with null safety, created and managed by JetBrains; popular in the Android community

Hybrid languages that compile to bytecodes

JRuby

Compiles Ruby

Jython

Compiles Python

Installing Groovy

The home page for Groovy is <http://groovy-lang.org>.

There are multiple options for installing Groovy.

- Zip download
- Windows installer
- Platform-specific installer (i.e., Homebrew or Macports on OS X)

In each case, all you need to do is:

1. Unzip the distribution to a common location
2. Set a `GROOVY_HOME` environment variable to that folder
3. Add `$GROOVY_HOME/bin` (or `%GROOVY_HOME%\bin` on Windows) to your PATH

These all assume that you have a `JAVA_HOME` environment variable set to the full JDK distribution (not just a JRE).

Using SDKMan, the Software Development Kit Manager

If your system supports a bash shell, consider using SDKMan, the Software Development Kit Manager, to install Groovy, Grails, Gradle, and other projects from the Groovy ecosystem. It is located at <http://sdkman.io>.

You install it using:

```
> curl -s get.sdkman.io | bash
```

Then to install the latest versions of Groovy and Grails:

```
> sdk install groovy
> sdk install grails // or gradle, or several others
```

This tool makes installs very simple, and allows you to switch between versions with a single command.

```
> sdk list groovy
// shows all available Groovy versions, default is 2.4.12
```

```
> sdk use groovy 2.3.6
// Use Groovy 2.3.6 in this shell
```

If you don't already have version 2.3.6 installed, this command will install it, too.

While it is not common to switch Groovy versions, Grails applications are tied to a specific version of Grails. GVM makes it easy to switch Grails versions to accommodate that.

To find out what version of Groovy you are currently using, use the **groovy** command with the **-v** flag.

```
> gvm use groovy 2.4.12
> groovy -v
Groovy Version: 2.4.12 JVM: 1.8.0 Vendor: Oracle Corporation OS: Mac OS X
```

If you are using Java 8 (JDK 1.8), you need Groovy 2.3+. All versions of Groovy support Java 6 and 7.

Hello, World

The "Hello, World" program in Java is quite verbose, involving multiple object-oriented concepts.

"Hello, World" in Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

To understand this, a Java developer has to know:

- **public** (and other access modifiers)
- what a **class** is
- what a **method** is
- what the **main** method means in Java
- the **dot notation** for accessing attributes and methods
- what an array of **String** references means
- the **braces notation** for delimiting code
- what a **static** method is

Developers from a Java background know all this intuitively, but there is a significant learning curve involved for non-Java people.

Here is the "Hello, World" program in Groovy:

"Hello, World" in Groovy

```
println 'Hello, World!'
```

Later sections will discuss this in more detail.

Some Groovy features:

- Semicolons are optional

Semicolons work if you add them, and are necessary if you have more than one statement on a single line, but are generally not needed.

- Parentheses are optional until they're not

That sounds silly, but it's true. If the compiler can guess correctly where the parentheses would have gone, you can leave them out. Otherwise, add them in. Groovy's goal is not to make the shortest code possible — it's to make the simplest code possible that's still understandable.

- Groovy is optionally typed.

If you declare a variable with a type, it works. If you don't know or don't care, you can use the `def` keyword discussed in a later section.

Running Groovy

The `groovyc` and `groovy` commands

In Java, you compile with the `javac` command and run with the `java` command.

```
> javac HelloWorld.java
// creates HelloWorld.class
```

```
> java HelloWorld
Hello, World!
```

Groovy has `groovyc` for compiling and `groovy` for execution, but you can actually execute the source code.

hello_world.groovy

```
println 'Hello, World!'
```

Use the `groovy` command to run the source:

```
> groovy hello_world.groovy
Hello, World!
```

The source is actually compiled, but the compiled version is not kept. That's interesting, but not very common.

In practice, Groovy code is compiled, especially as part of a larger project.

```
> groovyc hello_world.groovy
// creates hello_world.class
```

```
> groovy hello_world
Hello, World!
```



The Groovy compiler knows how to compile Java, too.

Since the contents of `hello_world.groovy` are not a class, the file is called a `script`. Groovy developers often use lowercase with underscores for script filenames. This makes it easy to see which files in a project are scripts and which are classes.

One little-used option is the `-e` flag on the `groovy` command, which executes the next argument as a

complete script.

```
> groovy -e 'println InetAddress.getLocalHost'
// prints name and IP address of host system
```

The Groovy Shell

Groovy includes an interactive REPL (Read-Eval-Print-Loop) called **groovysh**. It provides tab completion, history, and line-by-line execution.

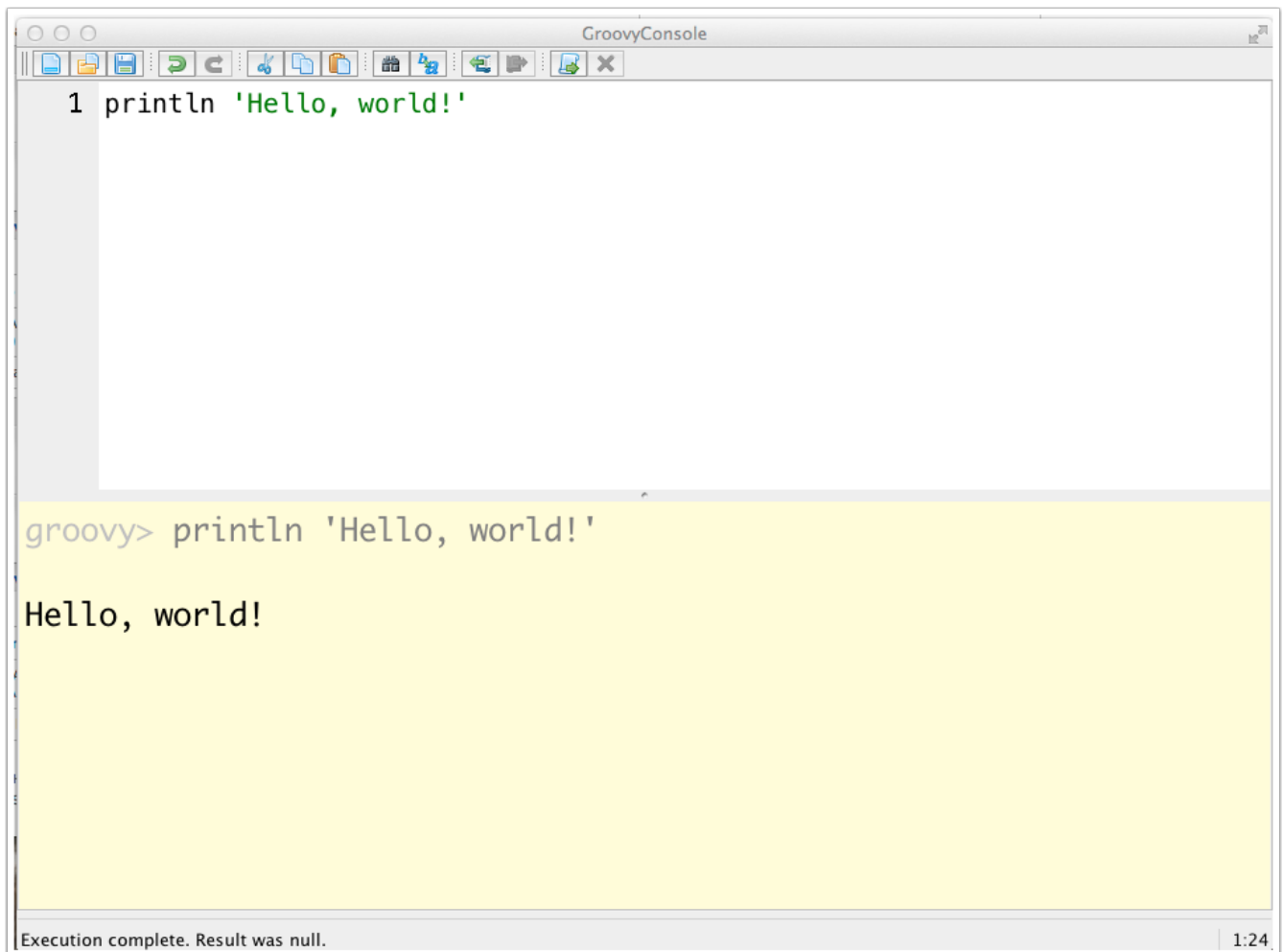
```
> groovysh
Groovy Shell (2.4.3, JVM: 1.8.0)
Type ':help' or ':h' for help.
-----
groovy:000> 1 + 1
==> 2
groovy:000> println 'Hello, World!'
Hello, World!
==> null
groovy:000> void sayHello(String name) {
groovy:001> println "Hello, $name!"
groovy:002> }
==> true
groovy:000> sayHello('Dolly')
Hello, Dolly!
==> null
```

Type **:help** or **:h** to see all available commands. Exit the Groovy shell using the **:exit**, **:quit**, **:x**, or **:q** commands.

The Groovy Console

The Groovy Console is a graphical user interface that allows you to execute Groovy code quickly and easily.

The **groovyConsole** command starts up the console.



Enter code in the top region and execute it by typing **Ctrl-Enter** or **Ctrl-R** on Windows or **Cmd-Enter** or **Cmd-R** on a Mac. There is also an *Execute Groovy Script* icon second from the end on the right.

The Groovy console is great for testing ideas when you don't want to start up a full-scale IDE.

Numbers

Groovy uses the same numeric types as Java, but there are no primitives in Groovy. Instead, Groovy uses the wrapper classes (like `Integer`, `Double`, and `Boolean`) for any primitive values.

The default type for a non-floating point literal is **Integer** or longer:

Non-floating point literals

[illegible]

Groovy uses `BigDecimal` for all floating-point literals and arithmetic calculations

Floating-point values

```
(2.5).getClass() == java.math.BigDecimal
(2/3).getClass() == java.math.BigDecimal
```

Unlike Java, division is done at `BigDecimal` levels even if both arguments are integers. If you want integer division, use the `intdiv` method in `java.lang.Number`.

assert, imports, and def

assert

Java has an `assert` keyword that is rarely used and turned off by default.

Groovy uses a "power assert" that returns a lot of information when it fails.

```
int x = 3
int y = 4
assert 7 == x + y // true, so returns nothing
assert 7 == x + y + 1
```

Assertion failed:

```
assert 7 == x + y + 1
|  |  |  |  |
| 3 7 4 8
false
```

```
assert 7 == x + 2 * y / (3**x - y) + 1
Assertion failed:
```

```
assert 7 == x + 2 * y / (3 ** x - y) + 1
|  |  |  |  |  |  |  |  |  |
| 3  | 8 4  | 27 3  | 4 4.3478260870
false|      |      23
|      0.3478260870
3.3478260870
```

All that extra debugging information means most Groovy developers use `assert` in all their tests

Automatic imports

In *Java*, if you don't add any `import` statements you get `java.lang.*` automatically.

In *Groovy*, if you don't add any `import` statements, you get:

- `java.lang.*`
- `java.util.*`
- `java.net.*`
- `java.io.*`
- `java.math.BigInteger`
- `java.math.BigDecimal`
- `groovy.lang.*`
- `groovy.util.*`

Groovy classes therefore have far fewer import statements than Java classes.

The **def** keyword

Groovy is optionally typed. If you declare a variable to be of type **String**, or **Date**, or **Employee**, then that's all that can be assigned to them.

```
Integer x = 1
x = 'abc' // throws ClassCastException
```

The **def** keyword tells the compiler we are not declaring a type, which will be determined at runtime.

```
def x = 1
assert x.getClass() == java.lang.Integer

x = new Date()
assert x.getClass() == java.util.Date

x = 'abc'
assert x.getClass() == java.lang.String
```

In Grails, properties in domain classes—which are mapped to database tables—require actual data types. The **def** keyword is often used as the return type on controller methods, however.

Strings and Groovy Strings

Groovy has two types of strings, single- and double-quoted.

Single-quoted strings are instances of `java.lang.String`.

Double-quoted strings are Groovy strings and allow interpolation:

```
def s = 'this is a string'
assert s.getClass() == java.lang.String

s = "this uses double-quotes but is still a String"
assert s.getClass() == java.lang.String

s = "this string uses interpolation: ${1 + 1}"
assert s == 'this string uses interpolation: 2'
assert s instanceof GString
```

The `${...}` notation inside a double-quoted string evaluates its contents as a Groovy expression and invokes `toString` on the result.

If you are evaluating a variable only, you can leave out the braces.

```
String first = 'Graeme'
String last = 'Rocher'
assert "$first $last" == 'Graeme Rocher'
```

Groovy also supports multiline strings. Three single quotes are regular multiline strings.

```
def picard = '''
Oh the vaccuum outside is endless
Unforgiving, cold, and friendless
But still we must boldly go
Make it so, make it so, make it so!
'''
```

This string has five lines, because the first line starts with a carriage return.

Three double-quotes are multiline Groovy strings.

```
def saying = """
There are ${Integer.toBinaryString(2)} kinds of people in the world
Those who know binary, and those who don't
"""
```

Multiline strings are helpful in many situations, but they are particularly useful when executing

SQL statements.

Finally, Groovy supports what are called *slashy* strings, which are delimited by forward slashes.

Slashy strings for regular expressions

```
def zip = /\d{5}(-\d{4})?/  
assert '12345' ==~ zip  
assert '12345-1234' ==~ zip  
assert '12345 12345-1234 1234'.findAll(zip) ==  
    ['12345', '12345-1234']
```

Slashy strings do not require you to use double backslashes inside regular expressions. Here the `\d` pattern represents a decimal digit. The pattern `\W` means any non-word character (i.e., other than `[a-zA-Z0-9_]`).

```
def testString = 'Flee to me, remote elf!'.toLowerCase().replaceAll(/\W/, '')  
assert testString == 'fleetomeremoteelf'  
assert testString == testString.reverse() // test string is a palindrome
```

The `==~` operator checks for an exact match and returns the boolean true or false. The `==` operator returns an instance of `java.util.regex.Matcher`.

Using a tilde on a slashy string turns it into an instance of `java.util.regex.Pattern`, as in

```
assert ~/abcd/ instanceof java.util.regex.Pattern
```

A more detailed discussion of regular expressions in Groovy can be found at <http://groovy.codehaus.org/Regular+Expressions>.

Operator Overloading

In Java, the only overloaded operator is `+`, which means addition for numerical values and concatenation for strings.

In Groovy, all operators invoke methods. The complete list of operators is given on <http://groovy.codehaus.org/Operator+Overloading>. Here is an abbreviated version.

Table 1. Table Operators and Corresponding Methods

Operator	Method
<code>a + b</code>	<code>a.plus(b)</code>
<code>a - b</code>	<code>a.minus(b)</code>
<code>a * b</code>	<code>a.multiply(b)</code>
<code>a / b</code>	<code>a.div(b)</code>
<code>a % b</code>	<code>a.mod(b)</code>
<code>a ** b</code>	<code>a.power(b)</code>
<code>a++</code>	<code>a.next()</code>
<code>a--</code>	<code>a.previous()</code>
<code>a[b]</code>	<code>a.getAt(b)</code>
<code>a[b] = c</code>	<code>a.putAt(b, c)</code>
<code>a == b</code>	<code>a.equals(b)</code> or <code>a.compareTo(b) == 0</code>

While overloading operators in regular Groovy programs is not that common, it's used throughout the standard libraries.

For example, the `java.lang.Number` class is the superclass of all the wrapper classes as well as `java.math.BigInteger` and `java.math.BigDecimal`. The Groovy JDK adds `plus`, `minus`, and others to `Number`, which allows the operators to be used with all of its subclasses.

POJOs vs POGOs

In Java, a Plain Old Java Object (POJO) is a class with attributes that normally have getters and setters.

Here is a typical Java example.

```
import java.util.Date;

public class JavaTask {
    private String name;
    private int priority;
    private Date startDate;
    private Date endDate;
    private boolean completed;

    public JavaTask() {
        // default constructor
    }

    public JavaTask(String name, int priority, Date start, Date end, boolean
completed) {
        this.name = name;
        this.priority = priority;
        this.startDate = start;
        this.endDate = end;
        this.completed = completed;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setPriority(int priority) {
        this.priority = priority;
    }

    public int getPriority() {
        return priority;
    }

    public void setStartDate(Date start) {
        this.startDate = start;
    }

    public Date getStartDate() {
        return startDate;
    }
}
```

```

public void setEndDate(Date end) {
    this.endDate = end;
}

public Date getEndDate() {
    return endDate;
}

public void setCompleted(boolean completed) {
    this.completed = completed;
}

public boolean isCompleted() {
    return completed;
}

@Override
public String toString() {
    return "JavaTask{" +
        "name='" + name + '\'' +
        ", priority=" + priority +
        ", startDate=" + startDate +
        ", endDate=" + endDate +
        ", completed=" + completed +
        '}';
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;

    JavaTask javaTask = (JavaTask) o;

    if (completed != javaTask.completed) return false;
    if (priority != javaTask.priority) return false;
    if (endDate != null ? !endDate.equals(javaTask.endDate) : javaTask.endDate !=
null) return false;
    if (name != null ? !name.equals(javaTask.name) : javaTask.name != null) return
false;
    if (startDate != null ? !startDate.equals(javaTask.startDate) : javaTask
.startDate != null) return false;

    return true;
}

@Override
public int hashCode() {
    int result = name != null ? name.hashCode() : 0;
    result = 31 * result + priority;
    result = 31 * result + (startDate != null ? startDate.hashCode() : 0);
}

```

```

        result = 31 * result + (endDate != null ? endDate.hashCode() : 0);
        result = 31 * result + (completed ? 1 : 0);
        return result;
    }
}

```

The comparable Groovy POGO is shown next.

A Groovy POGO

```

import groovy.transform.Canonical

@Canonical ①
class GroovyTask {
    String name
    int priority
    Date startDate
    Date endDate
    boolean completed
}

```

① AST transformation, discussed later

Why is the POGO so much shorter?

In *Java*, if you don't add an access modifier (**public**, **private**, or **protected**), you get "package private", which means other classes in the same package can access it.

In *Groovy*, if you don't add an access modifier:

1. Attributes are private by default
2. Methods are public by default
3. Classes are public by default

In *Java*, if you don't add any constructors, the compiler adds a default, no-arg, constructor that does nothing

In *Groovy*, if you don't add any constructors, you get both a default and a "map-based" constructor that uses the attributes as keys, so the need for overloaded constructors goes away.

```

Task t1 = new Task()
Task t2 = new Task(name:'t2', priority:3)
Task t3 = new Task(name:'t3', startDate: new Date(), endDate: new Date(), completed:
true)

```

In *Java*, getters and setters must be added explicitly.

In *Groovy*, getters and setters are generated for any attribute without an access modifier.

1. Attributes marked `final` will only have getters and not setters
2. Attributes marked `private` or `public` will generate neither getters nor setters

That brings up a very common Groovy idiom:



When you access a property in a POGO, Groovy automatically invokes the getter or setter

For example, consider the following code:

```
Task t = new Task()  
t.name = 't0' ①  
t.setPriority(3) ②  
assert t.name == 't0' ③  
assert t.getPriority() == 3 ④
```

- ① Uses generated `setName` method
- ② Uses generated `setPriority` method
- ③ Uses generated `getName` method
- ④ Uses generated `getPriority` method

What about the `@Canonical` annotation? That's the subject of the next section.

AST Transformations

Groovy uses annotations to trigger **Abstract Syntax Tree** (AST) transformations.

The compiler reads the source file and creates an abstract syntax tree. Then the annotation causes it to modify the tree before finishing the compilation process. AST transformations are thus compile-time metaprogramming.

Consider a `Person` class with `firstName` and `lastName` properties

Using the inherited `toString` method

```
package entities

class Person {
    String firstName
    String lastName
}

Person p = new Person(firstName: 'Guillaume', lastName: 'Laforge')
println p
```

The result will be something like `entities.Person@7996eb1d` (the hex address after the `@` sign will differ)

Adding an override of the `toString` method works just as it does in Java:

With an explicit `toString` method

```
package entities

class Person {
    String firstName
    String lastName

    String toString() { "$firstName $lastName" }
}

Person p = new Person(firstName: 'Guillaume', lastName: 'Laforge')
assert p.toString() == 'Guillaume Laforge'
```

The `@ToString` transformation

The annotation `groovy.transform.ToString` triggers an AST transformation that generates a `toString` method.

Adding the `@ToString` AST transformation

```
package entities

import groovy.transform.*

@ToString
class Person {
    String firstName
    String lastName

    String toString() { "$firstName $lastName" }
}

Person p = new Person(firstName: 'Guillaume', lastName: 'Laforge')
assert p.toString() == 'Guillaume Laforge'
```

The AST transformation did nothing in this case, because the source already had a `toString` method.

That's a good general rule:



Groovy will not replace an existing method

If you don't like what Groovy is generating, add your own implementation. Groovy will not change it.

To see the effect of the `@ToString` transformation, remove the existing `toString` method

Using the `@ToString` AST transformation

```
package entities

import groovy.transform.*

@ToString
class Person {
    String firstName
    String lastName
}

Person p = new Person(firstName: 'Guillaume', lastName: 'Laforge')
assert p.toString() == 'entities.Person(Guillaume, Laforge)'
```

The generated `toString` method gives the fully-qualified class name followed by the values of the attributes, in order from top down in the class.

If you check the GroovyDocs for `groovy.transform.ToString`, you'll find that it takes an optional property called `includeNames`.

Using `@ToString` with `includeNames`

```
package entities

import groovy.transform.*

@ToString(includeNames = true)
class Person {
    String firstName
    String lastName
}

Person p = new Person(firstName: 'Guillaume', lastName: 'Laforge')
assert p.toString() == 'entities.Person(firstName: Guillaume, lastName: Laforge)'
```

Many AST transformations have optional properties. Check the documentation of the associated annotations for details.

The `@EqualsAndHashCode` transformation

Another transform is provided by `@EqualsAndHashCode`

Adding `@EqualsAndHashCode`

```
package entities

import groovy.transform.*

@ToString(includeNames = true)
@EqualsAndHashCode
class Person {
    String firstName
    String lastName
}

Person p1 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
Person p2 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
assert p1 == p2
assert p1.hashCode() == p2.hashCode()
```

The transformation generates an `equals` method and a `hashCode` method by the prescription laid down by Joshua Bloch in his *Effective Java* book. The same methods are also generated by most IDEs, including Eclipse and IntelliJ IDEA.

You can customize the generated methods using the `includes` or `excludes` optional properties. See the docs for details.

The `@TupleConstructor` transformation

The `@TupleConstructor` transformation adds a constructor to the class that takes each attribute in order from top down.

Adding `@TupleConstructor`

```
package entities

import groovy.transform.*

@ToString(includeNames = true)
@EqualsAndHashCode
@TupleConstructor
class Person {
    String firstName
    String lastName
}

Person p1 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
Person p2 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
Person p3 = new Person('Guillaume', 'Laforge') ①
assert p1 == p3
```

① Generated constructor

In addition to optional properties like `includes` and `excludes`, this transform also allows `includeFields`, `callSuper`, and more. See the docs for details.

The `@Canonical` transformation

The combination of `@ToString`, `@EqualsAndHashCode`, and `@TupleConstructor` is so popular that it has a shortcut.

The `@Canonical` transformation is equivalent to all three.

```
package entities

import groovy.transform.*

@Canonical
class Person {
    String firstName
    String lastName
}

Person p1 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
Person p2 = new Person(firstName: 'Guillaume', lastName: 'Laforge')
Person p3 = new Person('Guillaume', 'Laforge')
assert p1.toString() == 'entities.Person(Guillaume, Laforge)'
assert p1 == p3
```

The sparseness of the code is remarkable. In five lines, this code has:

- private attributes
- public getter and setter methods
- default constructor
- tuple constructor
- map-based constructor
- a `toString` override
- an `equals` override
- a `hashCode` override

That's a lot of power for very little code.

There are many other AST transformations in the Groovy standard library. They include `@TypeChecked` and `@CompileStatic` for type safety, `@Sortable` to automatically implement sorting, `@Immutable` for generating unmodifiable objects, and `@Delegate` for implementing composition.

Collections

Lists

Groovy has native support for collections. To create a list of strings in Java, you would need to write code like:

Creating a list of strings in Java

```
import java.util.*

List<String> strings = new ArrayList<>();
strings.add("This"); strings.add("is");
strings.add("a"); strings.add("list");
strings.add("of"); strings.add("strings");
```

In Groovy, square brackets (`[]`) represent a list, whose default implementation is a `java.util.ArrayList`. Therefore the Groovy equivalent is:

Creating a list of strings in Groovy

```
def strings = ['This', 'is', 'a', 'list', 'of', 'strings']
assert strings.class == java.util.ArrayList
```

Changing the implementation type can be done in two ways. One is to use the `as` operator.

```
def strings = ['This', 'is', 'a', 'list', 'of', 'strings'] as LinkedList
assert strings.class == java.util.LinkedList
```

The `as` operator invokes the `asType` method.

Alternatively, you can replace the `def` declaration with the desired type.

```
LinkedList strings = ['This', 'is', 'a', 'list', 'of', 'strings']
assert strings.class == java.util.LinkedList
```

Groovy will then do the type conversion for you.



Groovy will try to convert any expression to the declared variable type.

This capability is powerful. For example, the `split()` method returns `String[]`, an array of strings. Converting that into a `List` is an annoying bit of code, which can be completely eliminated in Groovy.

Converting a `String[]` into a `Collection`

```
Collection strings = 'this is a list of strings'.split()
assert strings == ['this', 'is', 'a', 'list', 'of', 'strings']
assert strings.class == java.util.ArrayList
```

Sets

In Java, a set is a linear collection that is not ordered and does not hold duplicates.

Using a `Set`

```
def nums = [3, 1, 4, 1, 5, 9, 2, 6, 5] as Set
assert nums == [3, 1, 4, 5, 9, 2, 6] as Set
assert nums.class == java.util.LinkedHashSet
```

The order here is predictable because the elements are simple integers. Normally the results would not be in insertion order. The duplicates, however, have been eliminated.

The `java.util.SortedSet` interface sorts elements on insertion.

Using a `SortedSet`

```
def nums = [3, 1, 4, 1, 5, 9, 2, 6, 5] as SortedSet
assert nums == [1, 2, 3, 4, 5, 6, 9] as SortedSet
assert nums.class == java.util.TreeSet
```

Duplicates are determined using the `equals` and `hashCode` methods. As shown in a previous section, there is an AST transformation called `@EqualsAndHashCode` available, or you can use the `@Canonical` annotation which includes it.

```
import groovy.transform.*

@Canonical
class Person {
    String first
    String last
}

Person p1 = new Person(first: 'Graeme', last: 'Rocher')
Person p2 = new Person(first: 'Graeme', last: 'Rocher')
Person p3 = new Person('Graeme', 'Rocher')
Set people = [p1, p2, p3]
assert people.size() == 1
```

Collections have a `size` method, but Groovy makes that more general.



Groovy adds a **size** method to arrays, strings, collections, node lists, and more.

Maps

The native syntax for maps also uses square brackets, but separates keys from values using a colon in each entry.

Map example

```
def map = [a:1, b:2, c:2]
println map // [a:1, b:2, c:2]
println map.keySet() // [a, b, c]
println map.values() // [1, 2, 2]
```

The keys are assumed to be strings, unless they're numbers.

Groovy overloads the dot operator to be **put** or **get**, and the **putAt** and **getAt** methods are also available. Therefore you can write:

Map with overloaded operators

```
def map = [a:1, b:2, c:2]
map.d = 1 // overloaded dot
map['e'] = 3 // putAt method
map.put('f', 2) // Java still works
println map // [a:1, b:2, c:3, d:1, e:3, f:2]
```

Because the dot operator has already been overloaded in the class, you can't rely on it to convert properties to getter methods.

```
def map = [a:1, b:2, c:2]
println map.class.name // throws NullPointerException ①
assert map.getClass().name == java.util.LinkedHashMap
```

① First dot looks for a **String** key called **class** and returns null

Groovy's native syntax for lists, sets, and maps makes coding with those data structures easy. The subject of the next section, closures, shows how to process them.

Closures

A closure is a block of code including the referencing environment (see [https://en.wikipedia.org/wiki/Closure_\(computer_programming\)](https://en.wikipedia.org/wiki/Closure_(computer_programming)) for a formal definition). While the details of closures can get complex, Groovy makes them simple for the vast majority of cases.

Iterating with loops

Consider iterating over a list.

```
def nums = [3, 1, 4, 1, 5, 9]
```

The standard Java loop works in Groovy.

A standard Java for loop

```
for (int i = 0; i < nums.size(); i++) {  
    println nums[i]  
}
```

The so-called "for-each" loop introduced in Java 1.5 also works.

The Java for-each loop

```
for (int n : nums) {  
    println n  
}
```

You lose the index, but that's often not a major issue.

Groovy has a similar loop, called a "for-in" loop.

The Groovy for-in loop

```
for (n in nums) {  
    println n  
}
```

Processing Lists with Closures

All of these work, but none are the most common way to loop over a collection in Groovy. The most common technique is to use the `each` method, which takes an instance of `groovy.lang.Closure` as an argument.

Using `each` with a closure

```
nums.each { n -> println n }
```

The closure is the code inside the braces, `{ ... }`. Think of it as the body of a function without a name. The arrow symbol is used to separate the parameters to the function from the function body, which can be written over several lines.

A one-argument closure

```
nums.each { n ->
    println ""
    The 'each' method on a collection
    passes each element to a one-argument closure.
    Here the argument is called 'n' and its value is ${n}.
    ""
}
```

This is a highly contrived, not to mention verbose, example, but it works.

If you do not use the arrow symbol, the default dummy name for one-argument closures is called `it`.

Using the default arg name

```
nums.each { println it }
```



Groovy closures are assumed to take one argument by default. The default name of that argument is `it`

The `each` method is defined to take a one-argument closure. The `eachWithIndex` method takes a two-argument closure.

The `eachWithIndex` method takes a 2-arg closure

```
nums.eachWithIndex { val, idx ->
    println "nums[${idx}] == $val"
}
```

There are no default argument names for a multi-argument closure, so you must use an arrow to define them.

Java 8 lambdas

Java 8 introduced *lambdas* to the language, but they differ from Groovy closures in several significant ways.

- There is no class representing Java 8 lambdas.
- Lambdas can access external variables, but only if they either marked **final** or are "effectively final" (i.e., assigned once and never again). That does, however, make them free from side-effects.

Groovy closures are more versatile, but also more dangerous.

Groovy closures can access variables defined outside them. For example, here's way to sum the numbers (though admittedly not a good way):

Summing numbers (there are better ways)

```
int total = 0
nums.each {
    total += it
}
println "Total = $total"
```



The Groovy JDK defines a **sum** method in `java.lang.Iterable`.

Though Groovy is not a functional language, it does define some functional capabilities. For example, the **collect** method transforms a collection into a new one by applying a closure to each element.

*Doubling elements of a collection using **collect***

```
def doubles = nums.collect { it * 2 }
assert doubles == [6, 2, 8, 2, 10, 18]
```

Groovy also defines a **findAll** method that returns all elements that satisfy a closure.

*Using **findAll** to filter elements*

```
assert [3, 9] == nums.findAll { it % 3 == 0 }
```

The `inject` method

From a functional point of view, `collect` and `findAll` are like `map` and `filter`. The method corresponding to `reduce` is called `inject`, which is a bit more complicated.

The `inject` method

```
def nums = [3, 1, 4, 1, 5, 9]
int total = nums.inject(0) { acc, n ->
    acc + n
}
assert total == 23
```

There is a lot going on here.

- The `inject` method takes two arguments, an initial value and a closure.



Whenever the last argument to a method is a closure, you can put the closure outside the parentheses

- The closure takes two arguments. The first is the value that comes from executing the closure during that loop. The second is assigned to each element of the collection.
- The initial argument to `inject` (here, the zero) is the first value for the `acc` argument, short for accumulator.

The spread-dot operator

There is a short-cut for `collect` if you only need to invoke a method on each element.

The spread-dot operator

```
def strings = ['this', 'is', 'a', 'list', 'of', 'strings']
assert strings.size() == 6 // apply size() to the list
assert strings*.size() == [4, 2, 1, 4, 2, 7] // apply size() to each element
```

Anything you can do with the spread-dot operator you can do with `collect`, but it can be a convenient short-cut.

Maps and closures

Consider a map of keys and values.

```
def map = [a:1, b:2, c:2]
```

The `each` method is defined on maps to take a closure, but the closure can take either one or two

arguments.

If the closure takes one argument, it is an instance of the `Map.Entry` class, which has `getKey` and `getValue` methods.

Iterating over a map with a one-arg closure

```
map.each {  
    println "map[${it.key}] == ${it.value}"  
}
```

Here accessing the `key` or `value` properties invokes the getter methods by the usual Groovy idiom.

If you use a two-argument closure, Groovy splits the keys and values for you.

Iterating over a map with a two-arg closure

```
map.each { k,v ->  
    println "map[$k] == $v"  
}
```

The names of the dummy variables (`k` and `v`) are arbitrary.

All of the map methods are overloaded in this manner. For example, the following transforms a map of keys and values into a list where the keys equal the values.

Convert a map into a list of key=value

```
assert map.collect { k,v -> "$k=$v" } == ['a=1', 'b=2', 'c=2']
```

Since the Groovy JDK adds a `join` method to `Collection` that takes a delimiter, here is an easy way to convert a map of parameters and values into a query string for a RESTful web service.

Creating a query string

```
assert map.collect { k,v -> "$k=$v" }.join('&') == 'a=1&b=2&c=2'
```

That construct frequently comes in handy.

More on closures (optional)

All the closures considered so far were arguments to pre-defined methods. You can define your own closures and invoke them, however.

Here is a closure representing an `add` method.

A closure for addition

```
def add = { x,y -> x + y }
```

The `add` variable is defined with the `def` keyword, but `Closure` would also be acceptable.

The closure takes two (untyped) arguments, `x` and `y`, and returns their sum. The `return` keyword is optional, because the last evaluated expression is returned automatically.

You invoke the closure by using its `call` method, or by simply providing the arguments in parentheses.

Invoking the `add` closure

```
assert add.call(3,4) == 7
assert add(3,4) == 7
assert add('abc','def') == 'abcdef'
assert add([3, 1, 4, 1, 5], [9, 2, 6, 5]) == [3, 1, 4, 1, 5, 9, 2, 6, 5]
```

In each case the closure invokes the `plus` method on `x` with argument `y`. If you check the Groovy JDK docs, you'll find that the `plus` method on a list takes another list and performs a union with it.

Once again, if the last argument to a method is a closure, you can place the closure after the parentheses.

The `downto` method on `Number`

```
10.downto(7) { println it } // most common Groovy idiom
10.downto 7, { println it } // works, but rare
10.downto(7, { println it }) // Java developer who only recently learned Groovy
```

In Grails, closures are used to initialize data, to enforce constraints, to customize mappings, and much more. Internally they are used to do some serious metaprogramming, but that's beyond the scope of this course.

Miscellaneous operators

Safe navigation

Use `?.` to safely navigate associations.

Safe navigation

```
class Department {
    Manager boss
}

class Manager {
    String name
}

def d = new Department(boss: new Manager(name: 'Mr Burns'))

assert d.boss.name == 'Mr Burns'

d = new Department()
assert d?.boss?.name == null
```

The expression `d?.boss?.name` asks if `d` is not null, which if true then evaluates `getBoss()`. Then it checks whether that result is not null, whereupon it calls `getName()`.

If either are null, it returns `null`.

Spaceship

Groovy has a spaceship operator, `<=>`.

Spaceship operator

```
assert 2 <=> 4 == -1
assert 4 <=> 4 == 0
assert 6 <=> 4 == 1
```

The operator uses the `compareTo` method to evaluate the expression and returns -1 if the left-side is less than the right, 0 if they are equals, and +1 if the left-side is greater than the right.

The Groovy Truth

In Java, only boolean expressions can evaluate to true or false. Groovy generalizes this considerably.

In Groovy, all the following are true:

- Non-zero numbers
- Non-null references
- Non-empty strings
- Non-empty collections
- Regular expressions with a match
- Boolean `true`

This makes it much easier to get into trouble, of course, so test cases become more important than ever.

Elvis

Consider the ternary (three argument) operator in Java:

```
String name
String n = (name != null && name.size() > 0 ? name : 'World')
assert "Hello, $n!" == 'Hello, World!'
```

Because of the Groovy truth, if `name` is null or empty, it evaluates to false, so this can be simplified.

```
String name
String n = name ? name : 'World'
assert "Hello, $n!" == 'Hello, World!'
```

The question then becomes, why use `name` twice? What if we could just say, if `name` is not null and not empty (i.e. true by the Groovy truth) use it, otherwise use a default?

So remove the second instance of the variable:

```
String name
String n = name ?: 'World'
assert "Hello, $n!" == 'Hello, World!'
```

The expression `?:` is known as the Elvis operator, because someone with a vivid imagination looked at it on its side and thought he or she saw Elvis.

Yes, it's a reach, but a very useful operator.

Method References

Java 8 has a method reference operator which uses a double colon. For example, to refer to the `random` method in the `Math` class, you can write:

```
Stream.generate(Math::random)
    .limit(100)
    .forEach(System.out::println)
```

That will generate the first 100 random numbers (between 0 and 1) and print them. The method reference syntax is used both in `Math::random` and in `System.out::println`. Each refers to the method and supplies it as a lambda to the method. The `forEach` method, for example, takes a `Consumer` as an argument, which is one of the so-called *functional interfaces* defined in the API.

Groovy doesn't use the same syntax, but it has a similar technique. Groovy uses the `&` operator to convert a method into a closure.

Using the method reference operator

```
def visit(List elements, Closure closure) {
    elements.collect { closure(it) }
}
```

Here, the `visit` method takes a collection and a closure as arguments. The implementation of `visit` is to apply the closure to each element of the collection and return the transformed values. The idea is to provide a method that can traverse the collection, while letting the client specify what to do with each element.

One way to invoke the method is to use an explicit closure.

```
List strings = 'this is a list of strings'.split()

visit(strings) { println it }
```

This uses the standard Groovy idiom where, since the last argument is a closure, it's placed after the parentheses. The result is that the `visit` method prints each element.

Here's another example. Say you want the total length of all the strings. Then you can use this closure:

```
int total = 0
visit(strings) { total += it.size() }
println total
```

Say, however, that you want to use a method that already exists. You can use the method reference operator to convert the method into a closure and use then use it as the second argument.

```
def echo(obj) { println obj; obj }

visit(strings, this.&echo)
```

The `echo` method simply prints its argument and then returns it.

The result is:

```
this  
is  
a  
list  
of  
strings  
Result: [this, is, a, list, of, strings]
```

As an aside, experienced developers will recognize that the `visit` method is essentially a limited form of the *Visitor* design pattern, which separates the way to traverse a set of elements from what you want to do with them. Here the traversal is pretty trivial, because we're using a linear collection, but the same process would work for trees or other more complex data structures.

In languages like Groovy that support closures, many of the classic design patterns reduce to practically nothing.

Groovy and XML

Nothing demonstrates the gap between Java and Groovy better than XML.

Parsing XML

Consider the following XML document.

people.xml

```
<people>
  <person id="1">
    <name>Buffy</name>
  </person>
  <person id="2">
    <name>Willow</name>
  </person>
</people>
```

Say you want to parse this document and get the name of the second person. One Java solution is provided here.

```

import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import java.io.File;
import java.io.IOException;

public class ParsePerson {
    public static void main(String[] args) {
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        try {
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document doc = builder.parse(new File("people.xml"));
            NodeList names = doc.getElementsByTagName("name");
            Node secondNameNode = names.item(1);
            String secondName = secondNameNode.getFirstChild().getNodeValue();
            System.out.println("The second name is " + secondName);
        } catch (SAXException | IOException | ParserConfigurationException e) {
            e.printStackTrace();
        }
    }
}

```

To parse the document, you need a `parse` method, which is an instance method of `DocumentBuilder`. You get a `DocumentBuilder` from a `DocumentBuilderFactory`, which has a factory method called `newDocumentBuilder`. To get the `DocumentBuilderFactory`, you need to use its factory method, `newInstance`.

Once you've done all that, you have a `Document` instance. To find the element in question, the available search methods are either `getElementById` (but there are no id's) or `getElementsByTagName`. The latter returns a `NodeList`, which you can then use to get the `Node` you want.

Then, of course, you have to remember that the value of a `Node` is not the character data. The character data lives in the text node child of the node, so you need to use `getFirstChild` to get that child, and then, at long last, you can get the value by calling `getNodeValue`.

Here, on the other hand, is the Groovy solution:

get_second_name.groovy

```

def root = new XmlSlurper().parse('people.xml')
println "The second name is ${root.person[1].name}"

```

The difference is almost unfair.

Generating XML

Generating XML using Java is too horrible to include here. Using Groovy is almost trivial. To do that, use a `groovy.xml.MarkupBuilder`.

generate_people.groovy

```
def builder = new groovy.xml.MarkupBuilder()
builder.people {
  person(id:1) {
    name 'Buffy'
  }
  person(id:2) {
    name 'Willow'
  }
}
```

The result is the XML data from above.

Builders work through Groovy metaprogramming. They intercept "missing" methods (like `person` and `name`) and, in this case, turn them into DOM elements. If the "pretended" method takes an argument with a colon, it becomes an attribute of that element. If it takes a regular argument, it becomes the character data. If there is a closure, it becomes a child element, and so on.

There are other "builder" classes in the standard library, like `SwingBuilder` and `AntBuilder`.

Groovy and JSON

Parsing JSON data

Groovy includes a `JsonSlurper` class that can parse JavaScript Object Notation (JSON) data.

Consider a file with two people in it, in JSON format.

people.json

```
[
  { "name": "Buffy", "role": "slayer" },
  { "name": "Willow", "role": "witch" }
]
```

The `JsonSlurper` class has `parse` methods that take various sources, but to be different read in the data using the `getText` method in the `File` class, then parse it.

get_second_role.groovy

```
import groovy.json.*

String jsonTxt = new File('../people.json').text
def json = new JsonSlurper().parseText(jsonTxt)
assert json[1].role == 'witch'
```

Generating JSON

Likewise, building JSON data involves a builder class. In this case, it's `groovy.json.JsonBuilder`.

generate_json.groovy

```
import groovy.json.*

def builder = new JsonBuilder()
builder([name:'Buffy', role:'slayer'], [name:'Willow', role:'witch'])

assert builder.toString() ==
    '[{"name":"Buffy","role":"slayer"}, {"name":"Willow","role":"witch"}]'
```

The documentation of `JsonBuilder` is a bit thin. As with most open source projects, better documentation can be found in the test cases. If you look at the `groovy-core` repository at GitHub (<https://github.com/groovy/groovy-core>), there are several subprojects, one of which is `groovy-json`. The `JsonBuilderTest` class can be found there, in the `src/test/groovy` folder.

Grails

Grails Introduction

Grails is a complete-stack framework that uses convention over configuration.

Complete-stack

Everything is included, from the web server to the database and everything in between

Convention-over-configuration

Grails uses conventions instead of configuration files wherever possible. For example, all classes stored in the `grails-app/domain` folder in a Grails app are mapped to database tables

Grails uses Groovy DSLs to configure the Java projects Spring and Hibernate.

DSLs

Domain-Specific Languages. Common DSLs include music notation, chess notation, or ordering a complicated drink at Starbucks. Even SQL is arguably a DSL, since it is not a general-purpose language.

Spring

The Spring Framework, <http://spring.io>, is one of the most popular open source projects in the Java world. In Grails it is used to wire components together and manage transactions, among many other things.

Hibernate

The Hibernate framework, <http://hibernate.org>, is the leading Object-Relational Mapping (ORM) library in the Java industry.

Installing Grails

Grails does not use an installer. You do have to have a full Java JDK installed. A JRE alone is not sufficient.

To install Grails,

1. Download a zip file from <http://grails.org/Download> and unzip it to a convenient location.
2. Set a `GRAILS_HOME` environment variable to the unzipped location.
 - On Windows, use the *Environment Variables* button on the computer settings to assign `GRAILS_HOME` to wherever you have extracted the zip archive.
 - On Unix (including Macs), add something similar to the following to your `.profile`, `.bashrc`, `.zshrc`, or whatever settings file your preferred shell uses:

```
> export GRAILS_HOME=/path/to/grails
```

3. Add the `bin` folder under `GRAILS_HOME` to your `PATH` variable
 - On Windows, that's `set PATH=%GRAILS_HOME%\bin;%PATH%`

- On Unix, that's `export PATH=$GRAILS_HOME/bin:$PATH`

4. Type `grails -version` at a command prompt to verify the setup is correct.

Grails Links

The home page for Grails is at <http://grails.org>. This is where you find the downloads, the documentation, and other information.

The Grails User Guide is the first place you look for information. It is located at <http://grails.org/doc/latest/>.



If you are not using the latest version of Grails, change the word 'latest' to the version number to reload the User Guide for that version. For example, use <http://grails.org/doc/2.5.1/> for Grails version 2.5.1.

Starting a Grails project

Grails 3 uses Gradle as a build tool. Its own scripts (like `run-app`) all delegate to the underlying Gradle system.

If you are using the command line, type the word `grails` followed by the name of the command. A complete list is given in the Grails User Guide.

To start a new project, use the `create-app` command.

```
> grails create-app hello
```

Every Grails application has the same project structure.

If you are looking at a Grails application for the first time, the file `gradle.properties` in the root folder is a good place to start.

Here is a sample:

gradle.properties

```
grailsVersion=3.3.2
grailsWrapperVersion=1.0.0
gormVersion=6.1.8.RELEASE
gradleWrapperVersion=3.5
```

In this file,

- the `grailsVersion` property shows which version of Grails was used for this application. Every Grails app is tied to a particular version, and it's easier to change Grails version (at the command line) than it is to upgrade an app.

Fortunately, Grails generates a wrapper script, called `grailsw`, which you can use to run Grails.



On Un*x systems, you may need to make the `grailsw` script, via the command `chmod +x grailsw`

- the `gormVersion` property specifies which version of Grails Object Relational Mapping API is used by the app. GORM can change at a different rate than Grails.
- the `gradleWrapperVersion` number specifies which version of the underlying Gradle system is being used.

Project Structure

Looking at the project structure:

src

The folders `src/main/groovy` and `src/test/groovy` should be familiar to any Java developer. The former holds any Java or Groovy source code that is not already a Grails artifact (controllers, domain classes, services, and a few others) and will be added to the classpath. The folder `src/integration-test/groovy` is for, naturally enough, integration tests. Note that all tests in Grails use the Spock testing framework.

grails-app

This is where you will spend most of your time. Specifically:

grails-app/conf

Configuration files go here. The most useful file is likely to be `application.yml`, which corresponds to the same file in Spring Boot. It contains information about database connections, MIME types, default packages, and more.

grails-app/domain

Domain classes map to database tables.

grails-app/controllers

Controllers map URLs to methods (called *actions*) and redirect or forward to views.

grails-app/services

Services are transactional by default. They hold the business logic of the application.

grails-app/views

Subfolders of this directory are created for every controller and hold Groovy Server Pages (GSPs) representing the view layer of the application.

Other directories will be explored as the need arises.

Controllers

In the model-view-controller architecture, controllers receive HTTP requests and ultimately return HTTP responses.

Grails controllers map URLs to methods, known as *actions*.

Grails actions normally use `def` as their return type, for reasons discussed below.

Controllers are created using either the `create-controller` or `generate-controller` commands.

The `create-controller` command generates a controller with an empty `index` action.

```
> grails create-controller com.mycompany.Welcome
```

```
package com.mycompany

class WelcomeController {
    def index() {}
}
```

The `generate-controller` command builds the full "static scaffolding", which will be discussed later.

You do not need to use either wizard to create a controller class. If you make your own, be sure to save it in the `grails-app/controllers` directory (in package structure). The wizard also generates a unit test.

Accessing Controllers

The default URL mapping for Grails is:

```
http://<server>:<port>/appname/controller?action?id?
```

A question mark indicates optional parts of the URL.

- *appname* is the name of the application, here used as the context root
- *controller* is the name of the controller class, all lowercase, without the word `Controller` (i.e., `hello` for `HelloController`)
- *action* is the name of the method in the controller
- *id* is the `id` field of a domain class, if available or needed

For example, the URL:

```
http://localhost:8080/holygrails/task/show/2
```

invokes the `show` action in the `task` controller (`TaskController`) with an `id` of 2 in the `hollyrails` application.

Returning from Controller Actions

There are three basic ways to return from a controller action, here known as The Three R's:

- `render`
- `redirect`
- `return`

In Grails 2.3+, a `respond` action was added, which still does one of the three listed but also takes into account the MIME type in the request. There is also a `chain` action, but it is much less common.

`render`

Write directly to the output stream. Can write individual strings, or send an object into a view template (discussed further later). Also used to send non-view responses, like JSON or XML.

render

```
def greet() {  
    render "Hello, World!" // writes 'Hello, World!' to output stream  
}
```

`redirect`

Generates a new URL for the browser and sends it there. A new request is generated, so *all existing request parameters are lost*. Much like a `response.sendRedirect(...new URL...)` in a Java web application.

redirect

```
def index() {  
    redirect(action:'greet')  
}
```

This generates the new URL <http://localhost:8080/hello/greet> and sends the browser there.

`return`

If the last evaluated expression is a map, each of the entries is added as an attribute to the request. Then you are forwarded (using the same request) to a GSP with the same name as the action, in a folder with the same name as the controller, under `grails-app/views`

return

```
def hi() {  
    String n = params.name ?: 'World'  
    [person: n]  
}
```

If there is a `name` parameter in the request, this assigns it to `n`. Otherwise, `World` is assigned to `n`. Then Groovy map on the next line assigns the key `person` to the value of `n` and returns it. This adds the `person` attribute to the request and forwards to `grails-app/views/welcome/hi.gsp`.



Remember, the `return` key is optional in Groovy

After a `return`, the resulting GSP can access the added attributes directly.

Accessing a request attribute in a GSP

```
<html>
  <head>
    <title>Welcome</title>
  </head>
  <body>
    <h2>Hello, ${person}!</h2>
  </body>
</html>
```

The `params` map

Grails adds all input request parameters to a map called `params`. The values can be accessed like any other Groovy map.

```
params.name
params['name']
params.get('name')
```

You can also invoke methods, like `getMethod()`, on the map using the normal Groovy idiom

```
if(params.method == GET) { ... }
```

You can also use the `params` map as the argument to a constructor, part of the binding process:

```
Task task = new Task(params)
```

Any properties of the domain class whose names match keys in the map will be assigned automatically.

An equivalent process is to use:

```
task.properties = params
```

Either way, all matching properties are assigned.

Default Action in a Controller

The default action of a controller is invoked if the URL ends in the controller name.

- If a controller has *only one* action, it's the default.
- If a controller has *an action called `index`*, it's the default.
- If you set the `defaultAction` property in a controller, it's the default.

Grails Object Relational Mapping (GORM)

Grails uses Groovy metaprogramming to build custom domain-specific languages (DSLs) that configure Spring and Hibernate. Of particular interest is GORM, the Grails Object-Relational Mapping API.

By convention, every class in the `grails-app/domain` directory is mapped to a database table.

By default,

- the *class* name maps to a *table* name
- all *attributes* become *column* names in the table
- Grails adds an `id` attribute of type `Long` to the domain class is mapped to an auto-generated primary key value of the same name in the table
- a `version` column is added to the domain class and table, which is used for optimistic locking

Optimistic vs Pessimistic locking

If you need to lock a row, GORM adds a `lock` method to each domain object.

```
Quest q = Quest.get(1)
q.lock()
```

The lock is released when the Hibernate session completes or a transaction commits, whichever comes first.

This is known as pessimistic locking, because a lock is imposed even if there is no contention.

To avoid the lock overhead, Hibernate (and therefore Grails) implements optimistic locking through a `version` column in each database table.

The SQL type of the version column can be either a timestamp or an integer type, with the integer type being the default.

- Whenever a transaction commits on a table row, Hibernate automatically increments the value of the version column by one.
- Update methods generate SQL that contains a `where` clause with both the `id` and `version` properties in it.

This achieves locking behavior without an actual lock. If you are doing lots of reads and few writes, it works well. If too many writes occur, you're better off using pessimistic locking.

Logging SQL

The file `conf/DataSource.groovy` contains configuration information for the database(s) used by the

application.

To see the generated SQL, add the following to the top section of this file.

conf/DataSource.groovy

```
dataSource {
    // ... existing properties ...
    logSql = true
    formatSql = true
}
```

The `logSql` property writes SQL to the console. The `formatSql` property uses indentation and multiple lines to more easily read it.

```
Hibernate:
  insert
  into
    quest
    (version, date_created, last_updated, name)
  values
    (?, ?, ?, ?)
```

Optionally, you can also add `use_sql_comments` to the `hibernate` section of the same file. This adds a leading comment to each SQL command which may or may not be helpful.

Optional Hibernate-managed Properties

GORM automatically adds both an `id` property and a `version` property to each domain class, unless you prevent it. Two more properties are managed by Hibernate only if you add them explicitly.

The properties `dateCreated` and `lastUpdated` are set by Hibernate when an object is first saved and whenever it is modified.

Adding `dateCreated` and `lastUpdated`

```
class Quest {
    String name
    Date dateCreated
    Date lastUpdated
    // ... other properties
}
```

You must spell them that way and use the `java.util.Date` data type in order for Hibernate to manage them. Otherwise they are regular properties of the class and must be managed by the client.

Constraints

Each domain class has a static `constraints` block that is assigned to a closure. Inside this block *each attribute can be used as a method* to enforce constraints.

For example, to declare that the `name` of a quest can not be an empty string, set the `blank` constraint to `false`

Enforcing a `blank` constraint on `name`

```
class Quest {
    String name

    static constraints = {
        name(blank: false) // can leave out the parentheses
    }
}
```

Other constraints include:

Table 2. Some domain class constraints

Constraint	Meaning	Example
<code>blank</code>	String value can not be empty	<code>name blank: false</code>
<code>range</code>	Value must be within a Groovy range	<code>priority range: 1..5</code>
<code>inList</code>	Only selected values	<code>title inList: ['Mr', 'Mrs', 'Ms', 'Dr']</code>
<code>min, max</code>	Comparable value minimum or maximum	<code>price min: 0.0</code>
<code>minSize, maxSize</code>	Size of a string or collection	<code>tasks maxSize: 25</code>
<code>size</code>	Sets both min and max size	<code>age size: 18..65</code>
<code>nullable</code>	Allows a property to be null in the database (not null is the default)	<code>homePage nullable: true</code>
<code>creditCard, email, url</code>	Uses Apache Commons Validator routines	<code>homePage url: true</code>

To do custom validation, use the `validator` constraint. It is assigned to a closure containing one, two, or three arguments.

- For one argument, it is first argument is the value submitted
- For two arguments, the first is the value submitted and the second is a reference to the current object
- For three arguments, the first is the value submitted, the second is a reference to the current object, and the third is a reference to the Spring errors object

For example, to require two password fields to be equal:

```
static constraints = {  
    password1 validator: { val, user ->  
        val == user.password2  
    }  
}
```

Saving to the Database

Saving an object to the database is a three-step process:

1. *binding* → populate a domain object with input parameters
2. *validation* → verify properties satisfy constraints
3. *persistence* → save to database

Binding is done in a controller either with a constructor or by assignment:

Binding values to an object

```
Task t = new Task(params)  
// or  
t.properties = params
```

In either case, any domain class properties that match values in the `params` map are assigned automatically.

Validation can be done using the `validate` method, which returns a boolean.

```
Task t = new Task(params)  
assert t.validate()
```

If validation fails, the `validate` method also adds a Groovy map called `errors` to the domain object, whose keys are the properties that failed validation and whose values are the constraints that were violated. These are often used in test cases.

Failing validation

```
class TaskControllerTests {
  def 'blank name fails validation'() {
    when:
      Task t = new Task(name: ' ')

    then:
      !t.validate()
      'blank' == t.errors['name']
  }
}
```

The `validate` method is rarely used explicitly outside of tests, because saving an object invokes validation automatically.

Objects are saved using the `save` method, which returns the object or null.

The key is to remember that *save calls validate*.

- if `validate` returns `false`, `save` returns `null`
- if `validate` returns `true`, `save` returns the object

If you would prefer `save` to throw an exception when validation fails, use the `failOnError` argument

```
t.save(failOnError: true) // throws a ValidationException if invalid
```

Since we don't normally want to throw an exception whenever a user submits bad data, the most common idiom is to check for nulls.

Checking for null when saving

```
class TaskController {
  // ...
  def save() {
    Task t = new Task(params)
    if (!t.save()) {
      // send user back to form to fix inputs
    }
  }
}
```

Note that the save does not occur immediately, because of the way the Hibernate session works, which is the subject of the next section.

The Hibernate Session

When Enterprise Java Beans were first created, **entity** beans mapped classes to database tables. Entity beans were always connected, however, and since you don't want to hold open a database connection longer than absolutely necessary, you often defined additional classes called value object, or transfer objects, or even data transfer objects.

ProductEJB and ProductTO

```
class ProductEJB {
    String name
    BigDecimal price

    // ALWAYS connected
}

class ProductTO {
    String name
    BigDecimal price

    // NEVER connected
}
```

The pattern of having two classes with essentially identical properties is known as a code smell. When Hibernate came along, the designers decided that what was needed was a class of objects that were sometimes connected and sometimes not.

Product domain class (or entity)

```
class Product {
    String name
    BigDecimal price
}
```

Instances of this class are connected to the database during a Hibernate session.

The states of a Hibernate managed object are shown in the figure.

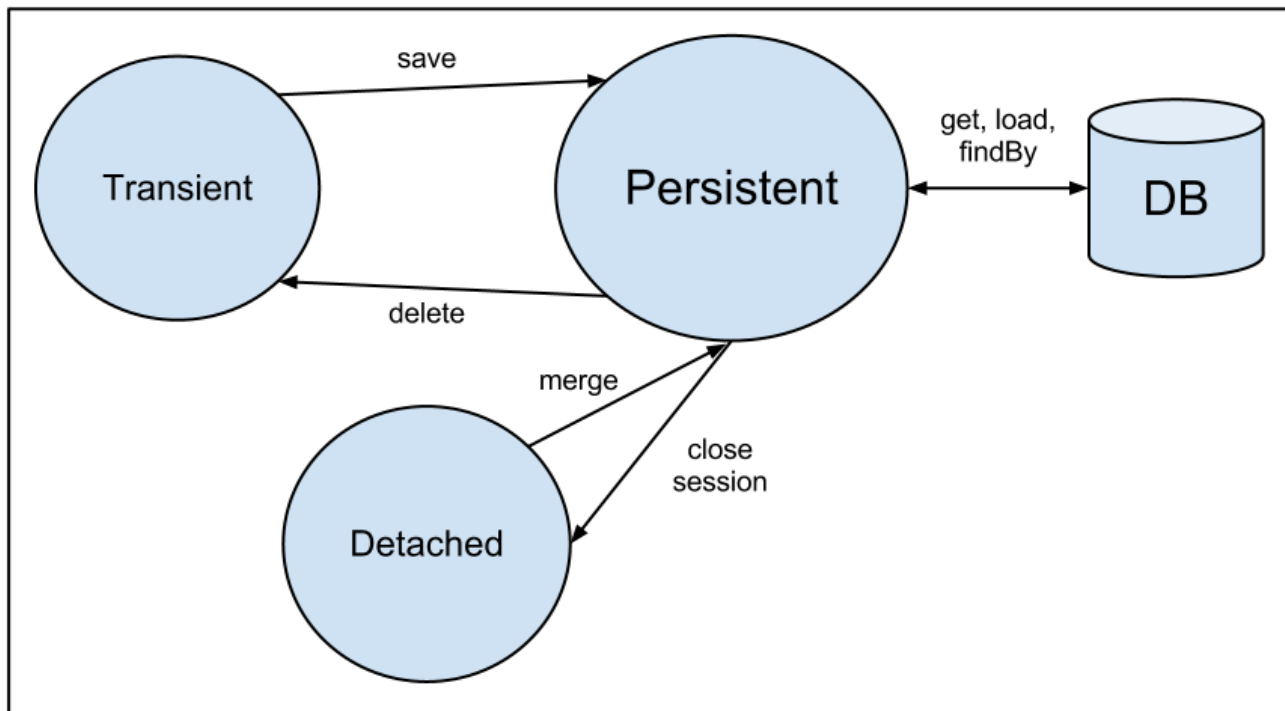


Figure 1. Hibernate States and Transitions

Objects managed by Hibernate are in one of three states:

Transient

An object is in memory, but has no corresponding row in the database. Newly instantiated objects are transient, as are objects that have been deleted from the database.

Persistent

An object retrieved using the Hibernate API (with methods like `get` or `findBy` or `load`) is persistent. *All changes to persistent objects are saved automatically.*

The collection of all persistent objects at any given moment is called the *persistence context* in JPA (the Java Persistence API).

Detached

When the Hibernate session closes or is flushed, objects are still in memory but no longer have their changes synchronized to the database.

How long does a Hibernate session last? It can be arbitrary, but Grails implements a common design pattern: *Open Session in View* (OSIV)

- On each request, a Spring interceptor begins a Hibernate session
- On each response, the interceptor closes the Hibernate session, thus flushing all changes to the database



The Open Session In View pattern means that each HTTP request is a single Hibernate session

The length of a Hibernate session is therefore measured in milliseconds, not minutes.

GORM Relationships

The Grails Object Relational Mapping API (GORM) establishes Hibernate relationships between domain classes.

One-to-Many

To establish a one-to-many relationship between two classes, use the `hasMany` property from GORM.

Quest.groovy

```
class Quest {
    // other attributes and methods not shown
    static hasMany = [tasks:Task]
}
```

This means:

- The `Quest` class now has an attribute called `tasks`
- The `tasks` attribute is of type `Set` (i.e., no duplicates) that holds only `Task` instances
- Grails adds instance methods `q.addToTasks(...)` and `q.removeFromTasks(...)` to the `Quest` class

There are four options for the `Task` side of the relationship.

1. Bi-directional, with cascade delete

```
class Task {
    // other attributes and methods not shown
    static belongsTo = [quest:Quest]
}
```

This means:

- The `Task` class has an attribute called `quest` of type `Quest`, so you can navigate from the task to its associated quest
- If the quest is deleted, all the associated tasks are deleted as well

2. Uni-directional, with cascade delete

```
class Task {
    static belongsTo = Quest
}
```

By assigning the `belongsTo` attribute to the class name, the cascade delete relationship is maintained, but there is no way to navigate from the task to the quest.

3. Bi-directional, NO cascade delete

```
class Task {  
    Quest quest  
}
```

You can navigate from the task to the quest, but when the quests are deleted, the tasks are not. Note that if the `quest` property is not nullable, this can cause referential integrity problems (discussed in the exercises)

4. Uni-directional, NO cascade delete

```
class Task {  
}
```

The task has no obvious association with the quest at all.

Many-to-Many

GORM supports many-to-many relationships, though there are alternative recommendations about them at the end of this section.

Consider a `Person` class and a `Movie` class. A person goes to many movies, and a movie has many people in the audience. One way to model this would be (*this is not correct yet*):

Person in a many-to-many relationship

```
class Person {  
    static hasMany = [movies: Movie]  
}
```

Movie in a many-to-many relationship

```
class Movie {  
    static hasMany = [people: Person]  
}
```

This is a bidirectional relationship, which in principle is fine, but actually introduces a problem. In the natural database mapping, this results in three tables rather than two.

```

create table Person (
    id bigint not null primary key,
    // other Person columns
)

create table Movie (
    id bigint not null primary key,
    // other Movie columns
)

create table Movie_Person (
    person_id // foreign key to Person
    movie_id  // foreign key to Movie
)

```

The problem is that if you add movie 2 to person 3, the same SQL is generated as if you add person 3 to movie 2. In other words, Hibernate would generate the same SQL twice.

Therefore, in a many-to-many relationship, you must designate one side or other as the *owner* of the relationship by putting a **belongsTo** attribute in the *other* side. Here, let's (arbitrarily) assume **Person** owns the relationship, so the proper definition of **Movie** is:

Movie with a belongsTo

```

class Movie {
    static hasMany = [people: Person]
    static belongsTo = Person
}

```

This means that you can only add and remove movies from person; you can no longer add and remove people from movies.



There is no cascade delete in a many-to-many relationship, even though one side or the other must have a **belongsTo** attribute

This does work, but the Grails team recommends that whenever you think you have a many-to-many relationship, consider breaking it into a pair of one-to-many relationships.

In this example, that becomes:

```
class Person {
  static hasMany = [tickets: Ticket]
}

class Movie {
  static hasMany = [tickets: Ticket]
}

class Ticket {
  Person person
  Movie movies
  Date when
  Theater theater
  // ... other attributes as necessary ...
}
```

Effectively, this has made a domain class out of the join table. That means you can add additional columns as needed. You can also use `belongsTo` in the `Ticket` class if you prefer.

Grails Queries

Grails has three different ways to create database queries: dynamic finders, criteria queries, and where queries.

Dynamic Finders

Dynamic finders have the form:

```
DomainClass.findBy([Property][Comparator][Boolean Operator])[Property][Comparator]
```

You can also use `findAllBy` with the same pattern.

Some examples:

Dynamic finders

```
Quest.findByName('Seek the grail')
Task.findAllByNameLike('%e%')
Task.findAllByNameLikeAndStartDateBetween(
    '%e%', new Date() - 1, new Date() + 1)
```

You can chain together as many properties and conditions as you like, but the method becomes increasingly unreadable as you do so.

Grails parses the query and generates the necessary SQL, then saves it internally using Groovy metaprogramming.

To see the SQL (and this is a good idea while developing), add properties to the `conf/DataSource.groovy` file.

Logging SQL in DataSource.groovy

```
dataSource {
    // ... other properties ...
    logSql = true
    formatSql = true
}
```

Now all SQL statements will be logged to the console. The `conf/Config.groovy` file allows you to use `log4j` instead and log to files.

Criteria Queries

Grails provides a great DSL for working with Hibernate criteria queries. Here is a simple example:

```
Task.withCriteria {  
    gt('priority', 2)  
    like('name', '%e%')  
    maxResults(5)  
    order('startDate', 'desc')  
}
```

For the H2 database built into Grails, this will generate SQL like:

```
Hibernate:  
    select  
        this_.id as id3_0_,  
        this_.version as version3_0_,  
        this_.completed as completed3_0_,  
        this_.end_date as end4_3_0_,  
        this_.name as name3_0_,  
        this_.priority as priority3_0_,  
        this_.quest_id as quest7_3_0_,  
        this_.start_date as start8_3_0_  
    from  
        task this_  
    where  
        this_.priority>?  
        and this_.name like ?  
    order by  
        this_.start_date desc limit ?
```

The `gt` and `like` keywords add to the `where` clause. The `maxResults` produces the `limit`, and the `order` method becomes the `order by` with the `desc`.

From the User Guide, `withCriteria` allows inline execution of criteria queries. The `createCriteria` command allows you to define a query before executing it.

The various methods, like `gt` and `like`, are found in the `org.hibernate.criterion.Restrictions` class of the Hibernate API, which can be found at <https://docs.jboss.org/hibernate/core/3.6/javadocs/org/hibernate/criterion/Restrictions.html>.

Where Queries

So-called **where** queries are another way of building up SQL select statements, but rather than have a custom DSL or break down complex method names, where queries use the actual properties of the domain object.

A sample where query

```
Task.findAll {  
  priority > 2 && name ==~ /.*e.*/  
}
```

Regular operators like `==` or `>` are self-explanatory. The `==~` generates a `like` clause and `=~` generates an `ilike` clause.

Used as the argument to a `find` or `findAll`, the query executes immediately. You can define them as detached queries using the `where` command and execute them later.

A sample where query

```
def query = Task.where {  
  priority > 2 && name ==~ /.*e.*/  
}  
  
query.list(sort: 'startDate')
```

Services

The need for a service layer

Here is the architecture of every Java web application ever (at least since Struts came out in 2001):

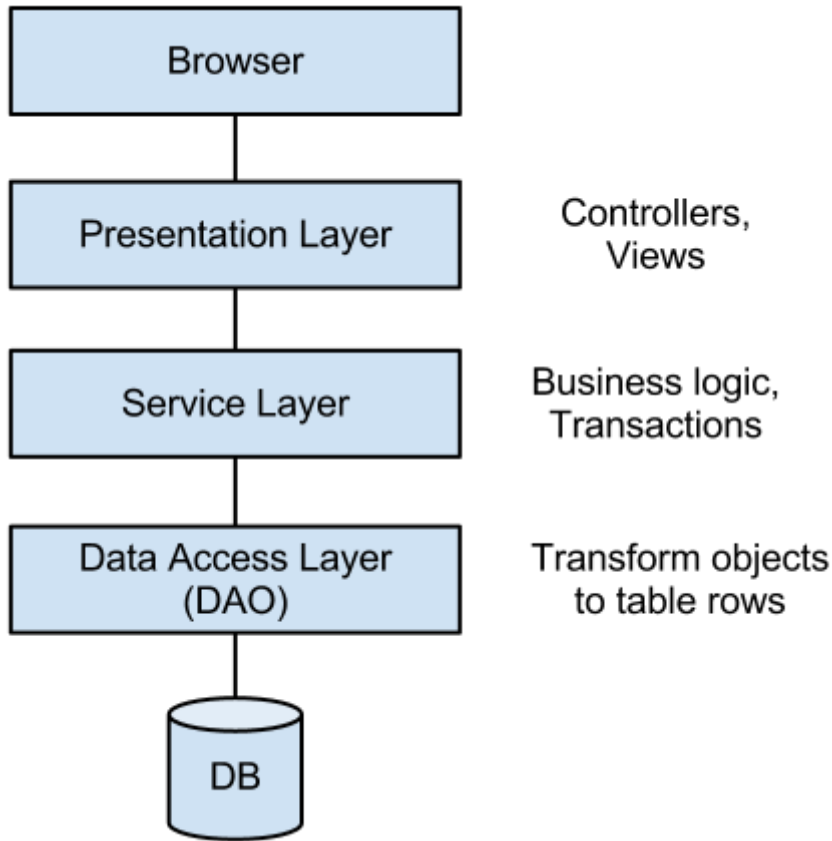


Figure 2. Architecture of Java Web Applications



The data access layer uses a design pattern known as Data Access Object

When you access the app through a browser:

1. The URL connects to a controller action
2. The controller accesses a service to do the work inside a transaction
3. The service pulls data from the DAO layer and updates and/or deletes it as requested
4. The controller then forwards or redirects the user to a new view

In non-Grails frameworks, you often write the DAO layer by hand. That means if you have a class called **Product**, you also have a class (or, better yet, an interface) called **ProductDAO** which contains the Create, Retrieve, Update, and Delete (CRUD) methods.

One possible `ProductDAO`

```
public interface ProductDAO {
    Product findById(Long id);
    List<Product> findAll();
    boolean add(Product p);
    boolean update(Product p);
    boolean delete(Long id);
}
```

This would be implemented in a class like `JDBCProductDAO` or `HibernateProductDAO` or even `JPAPProductDAO`.

While in principle you could make all the DAO methods transactional, that will cause complications when you need to use more than one DAO at a time. For example, if you were trying to buy a product, you might have a method like:

```
boolean checkOut(Product id, Customer id) {
    // look up product through ProductDAO
    // create a Shipping object using the ShippingDAO
    // create an Order using the OrderDAO
    // update the customer using the CustomerDAO
    // charge the credit card
    // Uh oh! Credit card failed! Roll everything back
}
```

The purpose of a service layer is to include methods like `checkOut`, which implement business logic and are transactional.

Grails and the DAO layer

Writing a DAO layer by hand can be very tedious (most of the required SQL is routine) and time consuming. It used to be estimated that as much as 40% of all project time was spent writing DAO classes.

In Grails, there are no explicit DAO classes. Grails uses Groovy metaprogramming to add the DAO methods to the domain classes. This is known as the **Active Record** design pattern.

Consider the `ProductDAO` interface in the earlier listing.

- `findById` becomes the static method `Product.get(id)`
- `findAll` becomes the static method `Product.list()`
- `add` becomes the instance method `product.save()`
- `update` is also a `product.save()`
- `delete` is the instance method `product.delete()`

In addition, Grails generates all the dynamic finder methods, and supports criteria queries and

where queries, all as static methods on `Product`. Groovy parses the method, generates the implementation including the Hibernate commands, and adds it to the domain class using metaprogramming.

This makes it very easy to access the database using methods on the domain classes. Unfortunately, that means it's easy to skip the service layer and call those methods directly from controllers, which means they're not inside transactions.

Grails without transactions

Consider the problem of deleting a quest that has both knights and castles. There is a cascade delete relationship between quest and task, so when we delete the quest, the tasks are deleted first.

If our `delete` method in the `QuestController` looks like:

```
def delete(Long id) {
    def q = Quest.get(id)
    if (!q) {
        flash.message = "Quest $id not found"
        redirect(action: 'list')
        return
    }
    try {
        q.delete(flush:true) // Loses data, ①
        flash.message = "Quest $id deleted"
        redirect(action: 'list')
    } catch (DataIntegrityViolationException e) {
        flash.message = "Referential integrity violation: Quest $id not deleted"
        redirect(action: 'show', id: id)
    }
}
```

① Looks like one method, but actually does 4 SQL commands

The problem with calling `q.delete()` is that it generates four SQL commands:

1. delete the first task
2. delete the second task
3. delete the third task
4. delete the quest -> fails because the knights reference it!

The knights have a foreign key that references the quest, so when we try to delete the quest, we get a referential integrity violation. Unfortunately, because we're not in an overall transaction, by the time this happens the tasks are already gone.

Grails services

Grails services are transactional by default.



Every method in a Grails service operates in a `REQUIRED` transaction

When a method is called in a Grails service, if there is no transaction already running, a new one starts. Then the method is invoked. When the method completes, the transaction either commits or rolls back, depending on what happened in the method.

If the propagation level is set to `REQUIRED` (the default), then if a transaction already is running when the method is called, it joins the transaction.

REST

REST stands for Representational State Transfer, an architectural style defined in a Ph.D. these by Roy Fielding in 2000. When applied to HTTP, REST consists of four principles.

Addressable resources

Every item in a system is accessible via a URL

Uniform interface

Only the HTTP verbs (primarily GET, POST, PUT, and , DELETE, but sometimes HEAD, OPTIONS, PATCH, and others) are allowed. No verbs should appear in a URL.

Content negotiation

Clients can request a particular *representation* of a resource, like JSON or XML. Typically this is done by specifying the MIME type as the value of the *Accept:* header in the request, but other mechanisms are acceptable as well.

Hypermedia

Hypermedia As The Engine Of Application State (HATEOAS) implies that each response contains links for subsequent requests. These links can be in the headers (known as *transitional* links) or in the body of the response (known as *structural* links).

REST in Grails (Grails 2.2 and earlier)

In Grails 2.2 and below, some of these capabilities are available.

The `grails.converters` package has both a `JSON` and an `XML` class, which can be used to transform domain objects into those types.

Transforming domain classes into XML format

```
import groovy.converters.XML

class CastleController {
    // ...
    def json() {
        render Castle.list() as XML
    }
}
```

The same mechanism works for JSON

```
import groovy.converters.JSON

class CastleController {
    // ...
    def json() {
        render Castle.list() as JSON
    }
}
```



The `grails.converters.deep` package is deprecated.

These both produce the default representations. If you need to customize them, then the `render` method takes a MIME-type as an optional argument.

Customizing JSON output

```
class KnightController {
    // ...
    def customjson() {
        render(contentType: 'application/json') {
            Knight.list().collect { k ->
                [id: k.id, name: k.toString(), quest: k.quest.name]
            }
        }
    }
}
```

The `conf/URLMappings.groovy` class also allows you to specify a `resource` so that the HTTP verbs get routed to controller actions.

REST in Grails (Grails 2.3 and above)

Grails 2.3 added significant new REST capabilities to Grails. A complete discussion is beyond the scope of these materials, but the basics are presented here.

Exercises

Chapter 1. Fundamentals

1. Strings and Groovy Strings

- a. Create a script called `strings.groovy`
- b. Add a statement to print a sentence, and execute the script

```
println 'this is a sentence'
```

- c. Use the `length` method from Java and the `size` method from Groovy to print the number of characters

```
String s = 'this is a sentence'  
println s.length()  
println s.size()
```

- d. Check programmatically that they give the same result

```
String s = 'this is a sentence'  
println s.length()  
println s.size()  
assert s.length() == s.size()
```

- e. Use array access (the `[]` operator) to print the first word.

```
// ... add to script ...  
println s[0..3]
```

- f. Print out the reverse of the first word

```
println s[3..0]
```

- g. Reverse the string using the array indices

```
println s[-1..0]
```

- h. Reverse the string using the `reverse` method from the Groovy JDK

```
println s.reverse()
```

- i. Use the `+` operator and check that it does not affect the original string

```
println s + ' with more words'
assert s == 'this is a sentence'
```

- j. Use the `-` operator to see how it works

```
println s - 'is'
```

- k. Chain minus operations together

```
println s - 'is' - 'is'
```

- l. Add a variable with your name and interpolate it into a Groovy string

```
String name = 'Dolly'
println "Hello, ${name}!"
```

- m. Try the same expression without the braces

```
String name = 'Dolly'
println "Hello, $name!"
```

- n. Try it with single quotes instead

```
String name = 'Dolly'
println 'Hello, $name!'
```

Chapter 2. Collections

1. Ranges

- a. Create a range from -3 to 3

```
def nums = -3..3
```

- b. Verify that it is inclusive

```
assert nums.contains(-3)
assert nums.contains(3)
```

- c. Invoke the `getFrom` and `getTo` methods using the normal Groovy idiom of accessing properties

```
assert nums.from == -3
assert nums.to == 3
```

- d. Create an open range and check its limits

```
nums = -3..<3
assert nums.contains(-3)
assert !nums.contains(3)
assert nums.from == -3
assert nums.to == 2
```

- e. Try a range of `Date` instances and the `format` method from the Groovy JDK

```
Date now = new Date()
Date then = now + 3
(now..then).each {
    println it.format('MMM dd, yyyy')
}
```

2. Lists

- a. Create a list of strings and check the default class

```
def strings = ['Red Sox', 'Yankees']
assert strings.class == java.util.ArrayList
```

- b. Use the left-shift and plus operators to append to the list

```

assert strings + ['Orioles', 'Blue Jays'] ==
  ['Red Sox', 'Yankees', 'Orioles', 'Blue Jays']
assert strings == ['Red Sox', 'Yankees']

strings << ['Orioles', 'Blue Jays']
assert strings == ['Red Sox', 'Yankees', ['Orioles', 'Blue Jays']]

```

- c. Use the minus operator to remove an element

```

assert strings - 'Yankees' ==
  ['Red Sox', ['Orioles', 'Blue Jays']]

```

- d. Try ranges and indices

```

println strings[0..2]
println strings[-3..-1]
println strings[1..-1]
println strings[0, 2]

```

- e. Use the `flatten` method

```

strings = strings.flatten()
assert strings == ['Red Sox', 'Yankees', 'Orioles', 'Blue Jays']
def nums = [3, [1, [4, [1, 5], 9], 2], 6]
assert nums.flatten() == [3, 1, 4, 1, 5, 9, 2, 6]

```

- f. Use the spread-dot operator and collect

```

assert strings*.size() == [7, 7, 7, 9]
assert strings.collect { it[0..2].toLowerCase() } ==
  ['red', 'yan', 'ori', 'blu']

```

- g. Use the `join` method

```

nums = nums.flatten()
assert nums.join(',') == '3,1,4,1,5,9,2,6'
assert nums.join('***') == '3***1***4***1***5***9***2***6'

```

3. Maps

- a. Start with an empty map

```

def map = [:]

```

- b. Add elements using dot operator, `putAt`, and `put` methods

```
def map = [:]
map.k1 = 'v1'
map['k2'] = 'v2'
map.put('k3', 'v3')

assert map.k1 == 'v1'
assert map['k2'] == 'v2'
assert map.get('k3') == 'v3'

println map.keySet()
assert map.keySet() == ['k1', 'k2', 'k3'] as Set
println map.entrySet()
println map.values()
assert map.values() as List == ['v1', 'v2', 'v3']
```

- c. Check the size of the collection

```
assert map.keySet().size() == 3
assert map.entrySet().size() == 3
assert map.size() == 3
```

- d. Create a query string from the map

```
assert map.collect { k,v -> "$k=$v" }.join('&') == 'k1=v1&k2=v2&k3=v3'
```

Chapter 3. Groovy Exercises

1. Sorting Strings

- a. Create a list of strings. Sort them alphabetically. Sort them by length. Sort them by length in descending order.



The Groovy JDK includes a `sort` method in the `java.lang.Iterable` class.

- b. *Advanced*: Sort by length, then sort equal length strings *reverse* alphabetically

2. Processing a list of numbers

- a. Create a list of numbers. Add them together. First double each number, then add them up. Compute their average.



The `java.lang.Iterable` class also has a `sum` method.

3. Reading a web page

- a. Using the Groovy JDK, access your home page and display the source code. Print the length of each line of the home page.



The Groovy JDK adds a `toURL` method to `java.lang.String` and a `getText` method to `java.net.URL`.

4. Closures as a filter

- a. Create a list of numbers. Print all elements greater than zero.

5. Multi-line strings

- a. Make a multi-line string. Compute the number of vowels on each line.



See the `findAll(String regex)` method in the `java.lang.String` class.

6. Padded binary output

- a. Print the numbers from 0 to 15 in binary (use Java's static `Integer.toString(num)` method). Use a method in `String` from the Groovy JDK to make all the output values have four digits.

Chapter 4. Grails Request Processing

1. Bookmarks

- a. Bookmark the following sites for future reference.

Groovy Homepage

<http://groovy-lang.org>

Groovy API

<http://groovy-lang.org/api.html>

Groovy JDK

<http://groovy-lang.org/gdk.html>

Grails Homepage

<http://grails.org/>

Grails User Guide

<http://docs.grails.org/latest/guide/single.html/>

2. A new Grails application

- a. Create a new Grails application called `hello` using the `grails create-app` command. Once it completes, change directory into the `hello` app.

NOTE: If you use the Grails Application Forge at <http://start.grails.org>, use the latest version of Grails and select the 'web' profile with all its defaults.

- b. Open the file `application.yml` in the `grails-app\conf` folder. Near the top you'll find a setting called `defaultPackage`, nested under `grails:` and `codegen:`. Change the value from `hello` (the default package name is the same as the app name) to `com.mycompany` (adjust the actual name as desired)
- c. Use the command `create-controller Welcome` to create a new controller called `com.mycompany.WelcomeController` (this should use the `defaultPackage` setting you changed in the previous step).
- d. In the `WelcomeController` class, what is the default action? What URL corresponds to this action? What would happen if we started the application and accessed this action now? (If you're not sure, try it and see.)

3. Render

- a. Inside the `index` action in the `WelcomeController` class, add a line rendering the phrase `Hello, World!`.

```
def index() {
    render "<h2>Hello, World!</h2>"
}
```

- b. Start the application (if not already started) using the `grails run-app` command and click on the `WelcomeController` link to access the action.
- c. Inside the `index` action, declare a `String` variable called `n`. Assign it to the value of the `name` property in the incoming request. If `params.name` is null or empty, default to the word `World`. Change the render to use the `n` value.



This is a perfect opportunity to use the Elvis operator from Groovy.

```
def index() {
    String n = params.name ?: 'World'
    render "<h2>Hello, $n!</h2>"
}
```

- d. To test the render operation, open the `WelcomeControllerSpec` class in the `hello/test/unit/com/mycompany/` directory. As of Grails 2.3, the default testing mechanism uses the Spock plugin, so the test extends `spock.lang.Specification`.
- e. The `setup` and `cleanup` methods are not required for this exercise. You can delete those methods or ignore them, whichever you prefer.
- f. Add tests for the `index` action both with an without a name. Use Spock's `when/then` construct to do so. Note that you can use the supplied `params` map to set the input `name` parameter before invoking the `index` action.

```
void "index without name should say Hello, World!"() {
    when:
        controller.index()

    then:
        response.text == '<h2>Hello, World!</h2>'
}

void "index with name should say Hello, name"() {
    when:
        params.name = 'Dolly'
        controller.index()

    then:
        response.text == '<h2>Hello, Dolly!</h2>'
}
```

4. Redirect

- a. Before implementing `redirect`, you need an action to redirect to. Add a new action to the `WelcomeController` called `greet`, which takes a single `String` argument called `name` and returns `def`. What is the URL corresponding to this action?
- b. Implement the `greet` action to use the `name` argument (note: only slight variation on current `index` implementation):

```
def greet(String name) {  
  String n = name ?: 'World'  
  render "<h2>Hello, $n!</h2>"  
}
```

- c. Test the `greet` action by revising the current tests for `index`. Note that you can supply the name as a method argument now, rather than a `params` value.

```
void "greet without name should say Hello, World!"() {  
  when:  
    controller.greet()  
  
  then:  
    response.text == '<h2>Hello, World!</h2>'  
}  
  
void "greet with name should say Hello, name"() {  
  when:  
    controller.greet('Dolly')  
  
  then:  
    response.text == '<h2>Hello, Dolly!</h2>'  
}
```

- d. Now you can re-implement the `index` action to redirect to the `greet` action.

```
def index() {  
  redirect(action:'greet')  
}
```

- e. Access the URL corresponding to the `index` action, *without* supplying a `name` parameter. Verify that you see the string 'Hello, World!' in the display.
- f. Add a `name` parameter to the URL for the `index` action and access the URL. Does the parameter get passed to the `hello` action? Why not?
- g. Add a second argument to the `redirect` method to pass the input parameters to the next request and try again. The supplied name should now show in the response.

```
def index() {
    redirect(action:'greet', params:params)
}
```

h. To test the redirect, add the following tests:

```
void "index without name should redirect to greet without name"() {
    when:
        controller.index()

    then:
        response.redirectedUrl == '/welcome/greet'
}

void "index with name should redirect to greet with a parameter"() {
    when:
        params.name = 'Dolly'
        controller.index()

    then:
        response.redirectedUrl == '/welcome/greet?name=Dolly'
}
```

5. Return

- Add a new action called `hi` to the `WelcomeController` class, with no arguments.
- This time, have the action return a map with one entry. The key in the entry should be `person` and the value is the `n` variable. Remember, the `return` keyword itself is optional. The complete implementation is shown below.

```
def hi() {
    String n = params.name ?: 'World'
    [user:n]
}
```

- Create a Groovy Server Page called `hi.gsp` in the `welcome` folder under `grails-app\views`. Give it a simple HTML structure (root element of `<html>`, with a `<head>` section and a `<body>` section). In the head section, add a `meta` tag to tell Grails to use the default SiteMesh layout.

In the body, add a `<div>` element with the CSS class `body`. Inside the `<div>`, add an `<h2>` element that displays "Hello, `${user}!`". The complete code is shown below.

```

<html>
  <head>
    <meta name="layout" content="main">
    <title>Greeting</title>
  </head>
  <body>
    <div class="body">
      <h2>Hello, ${user}!</h2>
    </div>
  </body>
</html>

```

- d. Start the server and access the `hi` action with a URL, once without a `name` parameter and once with one. Stop the server when you're done.
- e. Add two tests for the `hi` action, one with a name and one without. Check that the returned map has the proper `person` key in it.

```

void "hi without name should return map with user == 'World'"() {
  when:
  def model = controller.hi()

  then:
  model.user == 'World'
}

void "hi with name should return map with user == name"() {
  when:
  params.name = 'Dolly'
  def model = controller.hi()

  then:
  model.user == 'Dolly'
}

```

- f. Execute the `grails test-app com.mycompany.WelcomeController` and check the resulting output to make sure everything works.

Chapter 5. Creating A New Grails Application

1. Create the application

- a. Create an application called `holygrails` using the `web` profile with all the defaults. If you create the app from the command line, move into the application directory when you are finished.
- b. In `application.yml`, change the `defaultPackage` to your own domain. These materials will use `com.kousenit`, but as long as you use your own package consistently, any value is fine.

2. Add a domain class and a scaffolded controller

- a. Create a domain class called `Quest` using the `create-domain-class` wizard.
- b. Add an attribute called `name` of type `String` to the `Quest` class.

```
class Quest {  
    String name  
    // ...  
}
```

- c. Generate a controller and views using the `generate-all` command

```
> grails generate-all com.kousenit.Quest
```

3. Avoiding blank names

- a. Start the embedded Tomcat server:

```
> grails run-app
```

- b. Browse to the application home page, <http://localhost:8080> and select the `QuestController` link
- c. Create a new quest with any name you like (e.g., `Seek the grail`), edit it, and (optionally) delete it
- d. Try to create a quest with an empty name. Grails refuses to submit the form without a name. This is HTML5 client side validation.
- e. Type a few spaces in the `Name` field and submit again. You'll see that Grails gives an error that says the `name` field in `Quest` cannot be null. By default, Grails trims the input text, and if it is empty, the validator submits a null, and all fields in a domain class are marked NOT NULL in the database by default.
- f. Change this behavior by going back to `application.yml` and changing the following setting +

```
grails:
  databinding:
    convertEmptyStringsToNull: false
```

- a. Since the `application.yml` file is read only on startup, you'll need to restart the server. Do so, and see the new behavior when submitting a `Name` consisting of all spaces.
- b. In the `Quest` class, in the `constraints` section, add the restriction that the `name` cannot be blank:

```
static constraints = {
  name(blank: false)
}
```

- c. The Grails user guide shows that the default error key for blank constraints is `className.propertyName.blank`. In the `messages.properties` file in the `i18n` directory add the line:

```
quest.name.blank=Quests must have a name
```

- d. Re-submitting the quest with a blank name should give the new error message. (You may need to restart the server to see the new error message.)

Chapter 6. Adding Another Domain Class

1. Add a **Task** class

- Use the `create-domain-class` wizard to make a new domain class called **Task**
- Add the following attributes to the **Task**

```
class Task {  
    String name  
    int priority = 3  
    Date startDate = new Date()  
    Date endDate = new Date()  
    boolean completed  
}
```

- Add constraints to the **Task** properties

```
static constraints = {  
    name blank:false  
    priority range: 1..5  
    startDate()  
    endDate validator: { val, task ->  
        val >= task.startDate  
    }  
    completed()  
}
```

- Generate the controller and views for the **Task**

```
> grails generate-all com.kousenit.Task
```

2. Relationships

- Establish a one-to-many relationship between **Quest** and **Task** by adding to the **Quest.groovy** file:

```
static hasMany = [tasks:Task]
```

- Ensure that deleting a quest deletes all associated tasks by adding to **Task.groovy**:

```
static belongsTo = [quest:Quest]
```

- Re-start the server, add a quest, save it, then add a few tasks. Note that you can traverse from a quest to its tasks and from a task to the quest it belongs to. Delete the quest and note how the tasks also vanish.

- d. Add a `toString` method (which will override the one from `Object`) in `Quest`:

```
String name // (already there)
String toString() { name }
```

- e. Generate a `toString` method in `Task` using the AST Transformation:

```
import groovy.transform.ToString
@ToString(includes = ['name','priority','completed'])
class Task {
    // ...
}
```

3. Transient property

- a. Add a getter method for `duration` to `Task`

```
int getDuration() { endDate - startDate + 1 }
```

- b. Declare the `duration` property in a `transients` list

```
static transients = ['duration']
```

4. Optional properties

- a. Back in `Quest`, add Hibernate-managed properties

```
class Quest {
    Date dateCreated
    Date lastUpdated
    // ...
}
```



The `dateCreated` and `lastUpdated` fields do not display on the current scaffolding. They are in the database, however, which will be demonstrated in an upcoming exercise.

Chapter 7. Unit Testing Domain Classes

1. Unit test the Quest

- a. Open the file `QuestSpec.groovy` in the `src/test/groovy` hierarchy and add a `Quest` attribute with a non-null `name` value

```
// inside class QuestSpec
    Quest q = new Quest(name: 'name')
```

- a. Add a test to check that the quest attribute is valid

```
void "the default quest is valid"() {
    expect: q.validate()
}
```

- b. Add a test to check that an empty name fails as expected

```
void "blank names are not valid"() {
    when:
        q = new Quest(name: ' ')

    then:
        !q.validate()
        'blank' == q.errors['name'].code
}
```

- c. Execute the unit test and check the test report

```
> grails test-app com.kousenit.Quest
```

2. Unit test the Task

- a. Open the file `TaskSpec.groovy` and add attributes for a valid `Quest` and a valid `Task` (recall that we already initialized the `priority`, `startDate`, and `endDate` fields to valid values). The `@Shared` annotation is used when a property should not be re-initialized between tests (`@Shared` is in the `spock.lang` package)

```
@Shared Quest q = new Quest(name: 'q')
Task t = new Task(name: 'name', quest: q)
```

- b. After adding each of the following tests, verify that they pass using

```
> grails test-app com.kousenit.Task
```

c. Check the `getDuration` method

```
void 'check the default duration should be 1'() {  
    // default task has equal start and end dates  
    expect: 1 == t.duration  
}  
  
void 'check the duration calculation'() {  
    when: 'end date is 1 beyond start date'  
        t.endDate = t.startDate + 1  
  
    then: 'the duration is 2'  
        2 == t.duration  
}
```

d. Verify that the default task is valid

```
void 'default task is valid'() {  
    expect: t.validate()  
}
```

e. Test that a blank name fails as expected

```
void 'blank name is not valid'() {  
    when:  
        t = new Task(name: ' ', quest: q)  
  
    then:  
        !t.validate()  
        'blank' == t.errors['name'].code  
}
```

f. Check that priorities too low fail

```
void 'priorities below 1 are not valid'() {  
    when:  
        t.priority = 0  
  
    then:  
        !t.validate()  
        'range.toosmall' == t.errors['priority'].code  
}
```

g. Do the same for priorities too high

```

void 'priorities above 5 are not valid'() {
  when:
    t.priority = 6

  then:
    !t.validate()
    'range.toobig' == t.errors['priority'].code
}

```

- h. Check that valid priorities work properly using the cool `@Unroll` annotation from Spock. Note that you will need to add `import spock.lang.Unroll` to the set of existing import statements.

```

@Unroll
void 'task with priority #p is valid'() {
  when:
    t.priority = p

  then:
    t.validate()

  where:
    p << (1..5)
}

```

- i. Check that end dates before the start date fail

```

void 'endDate before startDate fails'() {
  when:
    t.endDate = t.startDate - 1

  then:
    !t.validate()
    'validator.invalid' == t.errors['endDate'].code
}

```

- j. In each case, run the unit tests and correct any errors

Chapter 8. Dynamic finders and criteria queries, part 1

1. SQL logging

- a. Turn on SQL logging by opening the file `grails-app/conf/application.yml` and adding:

```
dataSource:
  logSql: true
  formatSql: true
```

- b. Optionally, in the Hibernate section of the same file, add:

```
hibernate:
  use_sql_comments: true
```

2. Add initialization data

- a. Add some sample quests and tasks to the `BootStrap.groovy` file in the `grails-app/init` hierarchy

```
package holygrails

import com.kousenit.*

class BootStrap {
    def init = { servletContext ->
        Quest q = new Quest(name:'Seek the grail')
        .addToTasks(name:'Run away from killer rabbit')
        .addToTasks(name:'Answer the Bridge Keeper', priority:4)
        .addToTasks(name:'Defeat the Black Knight', completed:true)
        .save()
    }
}
```

3. The Grails Console

- a. If you are using IntelliJ IDEA, be sure to *stop the server*. The Grails console and the server run in the same thread and will run sequentially, meaning running the `console` command will look like it's not doing anything if the server is currently running.
- b. Start the Grails console

```
> grails console
```

- c. In the console, import the `com.kousenit` package

```
import com.kousenit.*
```

- d. Show all the quests

```
Quest.list()
```

- e. The result should show the single quest added in the bootstrap. Save a new, invalid quest

```
Quest q = new Quest().save()
```

- f. The result should be empty, even missing the SQL, because the Quest doesn't have a name. Make the save operation throw an exception if the quest is not valid by adding `failOnError:true` to the `save` method and see the resulting `ValidationException`

```
Quest q = new Quest().save(failOnError:true)
```

- g. Add the `failOnError:true` value to the `save` in the bootstrap. Note that this will not change anything until you restart the console.
- h. Verify that the quest and tasks have been saved

```
println Quest.list()  
println Task.list()
```

- i. Check the `id` and `name` properties of the `Task` instances

```
println Task.list().id  
println Task.list().name
```

- j. Verify that you can traverse from a task to its quest

```
println Task.get(1).quest.name
```

Chapter 9. Dynamic finders and criteria queries, part 2

1. Dynamic finders

- a. Continuing from the previous exercise, try a couple of dynamic finders (one at a time, or add `println` in front of each).

```
Quest.findByName('Seek the grail')
Task.findAllByPriorityGreaterThan(2)
Task.findAllByPriorityLessThanAndStartDateBetween(
    4, new Date() - 1, new Date() + 1)
```

- b. Try out the `count` method and others

```
Task.count()
Task.listOrderByName()
```

2. Criteria Queries

- a. Do a basic criteria query on `Task`

```
Task.withCriteria {
    ilike('name', '%e%')
}
```

- b. Add conditions

```
Task.withCriteria {
    ilike('name', '%e%')
    between('startDate', new Date() - 1, new Date() + 1)
    lt('priority', 4)
}
```

- c. Ordering works, too

```
Task.withCriteria {
    ilike('name', '%e%')
    between('startDate', new Date() - 1, new Date() + 1)
    lt('priority', 4)
    order('name', 'asc')
}
```

- d. Try a criteria query on `Quest` that uses a join

```

Quest.withCriteria {
  ilike('name', '%e%')
  tasks {
    ilike('name', '%e%')
    gte('priority', 2)
  }
}

```

3. Where queries

- a. Try a non-lazy where query

```

Task.findAll {
  priority > 2
}

```

- b. Add a like clause

```

Task.findAll {
  priority > 2 && name =~ '%e%'
}

```

- c. Use an `in` clause

```

Task.findAll {
  priority in [1, 3, 5]
}

```

- d. Define a lazy query and invoke it

```

def query = Task.where {
  priority > 2
}

query.list()

```

- e. Sort the results by name

```

def query = Task.where {
  priority > 2
}

query.list(sort:'name', order:'desc')

```

f. Can even do associations and sub-queries

```
def query = Quest.where {  
  name =~ '%e%'  
  tasks {  
    priority >= avg(priority)  
  }  
}  
  
query.list(sort: 'name')
```

Chapter 10. Complete the domain model

1. Knights

- a. Add a **Knight** domain class

```
> grails create-domain-class Knight
```

- b. Knights have titles and names

```
class Knight {  
    String title  
    String name  
}
```

- c. Add appropriate constraints

```
static constraints = {  
    title inList: ['Sir', 'Lord', 'Lady', 'King', 'Queen']  
    name blank: false  
}
```

- d. Quests have many knights, too

```
class Quest {  
    // ...  
    static hasMany = [tasks:Task, knights:Knight]  
}
```

- e. Knights know which quest they are on

```
class Knight {  
    Quest quest  
}
```

- f. Knights can be between quests

```
static constraints = {  
    // as above, and  
    quest nullable: true  
}
```

- g. Add a **toString** method to **Knight**

```
String toString() { "$title $name" }
```

- h. Generate a controller and views for **Knight**

```
> rails generate-all com.kousenit.Knight
```

2. Castles

- a. Add a **Castle** domain class

```
> rails create-domain-class Castle
```

- b. Castles are really just named locations

```
class Castle {  
  String name  
  String city  
  String state  
  double latitude  
  double longitude  
}
```

- c. Add appropriate constraints

```
static constraints = {  
  name blank:false  
  city blank:false  
  state blank:false  
  latitude min:-90d, max:90d // South Pole to North Pole  
  longitude()  
}
```

- d. Castles need a **toString** method, too

```
String toString() { "$name Castle" }
```

- e. Castles contain knights

```
static hasMany = [knights:Knight]
```

- f. Knights know their castles

```
class Knight {
  // as before
  Castle castle
}
```

g. But Knights can be between castles, too

```
static constraints = {
  // other Knight constraints
  castle nullable: true
}
```

h. Add a some castles and knights to the bootstrap data

```
def init = { servletContext ->
  // existing quest and tasks
  Castle camelot = new Castle(name:'Camelot', city:'Boston', state:'MA')
    .addToKnights(title:'King', name:'Arthur', quest:q)
    .addToKnights(title:'Sir', name:'Lancelot', quest:q)
    .addToKnights(title:'Sir', name:'Robin', quest:q)
    .save(failOnError:true)
}
```

i. Add another castle near by (will be useful later)

```
def init = { servletContext ->
  // others as above
  Castle swamp = new Castle(name:'Swamp', city:'Natick', state:'MA')
    .save(failOnError:true)
}
```

3. Generate a Castle controller

a. Generate the controller and views for the `Castle` class

```
> rails generate-all com.kousenit.Castle
```

4. Look at generated schema

a. Run the `schema-export` command

```
> rails schema-export
```

b. Check out the `ddl.sql` file in the `build` directory

c. Start the server

```
> rails run-app
```

d. Check out the db console at <http://localhost:8080/holygrails/dbconsole>. Be sure to use the same settings as those in the `dataSource` section of the `application.yml` file

Chapter 11. Create a Geocoder service

1. Google Geocoder Docs

- a. The Google Geocoder documentation is at <https://developers.google.com/maps/documentation/geocoding/>. It's probably worth a bookmark.

2. Creating a Service

- a. Create the geocoder service

```
> grails create-service Geocoder
```

- b. Add a public, static, final attribute representing the base URL for the geocoder to the `GeocoderService` class. Be sure to include the trailing question mark.

```
class GeocoderService {  
    public static final String BASE =  
        'http://maps.googleapis.com/maps/api/geocode/xml?'  
}
```

- c. Add a method to fill in the latitude and longitude values for a `Castle`.

```
Castle fillInLatLng(Castle c) {  
  
}
```

- d. Create an encoded address using a list and the collect and join methods.

```
void fillInLatLng(Castle c) {  
    String encoded = [c.city, c.state].collect {  
        URLEncoder.encode(it, 'UTF-8')  
    }.join(',')  
}
```

- e. Build a query string using a map and the encoded address

```
void fillInLatLng(Castle c) {  
    // from above...  
    String qs = "address=$encoded"  
}
```

- f. Transmit the URL to Google and parse the response XML.

```
// from above...  
def root = new XmlSlurper().parse("$BASE$qs")
```

- g. Update the latitude and longitude of the castle by walking the tree. Since occasionally Google returns bad data, don't try to parse the values unless you're sure they are present

```
// from above...  
def loc = root.result[0].geometry.location  
if (loc.lat.text()) {  
    c.latitude = loc.lat.toDouble()  
    c.longitude = loc.lng.toDouble()  
}  
c // or "return c"
```

3. Test the Geocoder service

- a. Open the `GeocoderServiceSpec` class and add the `DomainUnitTest<Castle>` trait. That will mock the `Castle` domain class when testing the service.

```
class GeocoderServiceSpec extends Specification implements  
    ServiceUnitTest<GeocoderService>, DomainUnitTest<Castle> {  
    // ...  
}
```

- b. Test for Google headquarters and for a non-existent location.

```

void "verify lat,lng for Mountain View, CA"() {
  given:
    Castle google = new Castle(name: 'Google', city: 'Mountain View', state: '
CA')

    when:
      service.fillInLatLng(google)

    then:
      (google.latitude - 37.4).abs() < 0.1
      (google.longitude - -122.1).abs() < 0.1
}

void 'service does not throw an exception on bad responses'() {
  given:
    Castle black = new Castle(name: 'Black', city: 'The Wall', state: 'Westeros
')

    when:
      service.fillInLatLng(black)

    then:
      black.latitude < 0.01
      black.longitude < 0.01
}

```

- c. Run the unit tests until the service test passes.

4. Injecting a service

- a. Inject the service into the castle controller by adding an attribute to the top of the class:

```

class CastleController {
  GeocoderService geocoderService
  // ...
}

```

- b. In the **save** action, just before the actual call to **save()**, update the castle

```

def save(Castle castle) {
  // before castleService.save() call
  geocoderService.fillInLatLng(castle)
}

```

- c. Do the same in the **update** action

```
def update(Castle castleInstance) {
    // before the castleService.save() call
    geocoderService.fillInLatLng(castle)
    // ...
}
```

- d. Inject the service into `BootStrap.groovy` as well

```
class BootStrap {
    GeocoderService geocoderService
    // ...
}
```

- e. Use the service to update the castles in the bootstrap before saving them

```
def init = { servletContext ->
    // ...
    Castle camelot = new Castle(name:'Camelot', city:'Boston', state:'MA')
        .addToKnights(title:'King', name:'Arthur', quest:q)
        .addToKnights(title:'Sir', name:'Lancelot', quest:q)
        .addToKnights(title:'Sir', name:'Robin', quest:q)
    geocoderService.fillInLatLng(camelot)
        .save(failOnError:true)
}
```

- f. Do the same for Swamp castle in the bootstrap

```
Castle swamp = new Castle(name:'Swamp', city:'Natick', state:'MA')
geocoderService.fillInLatLng(swamp)
    .save(failOnError:true)
```

5. Use the service in the application

- Restart the application and browse to the list of castles (access the castle controller)
- Add other castles, using any city and state you wish. Note that the state is interpreted broadly—you can use country codes, country names, or anything else that Google understands.

6. Disable client setting of lat/lng

- Open the file `grails-app/views/castle/create.gsp`
- Comment out (or delete) the `<f:all bean="castle"/>` tag
- Add the following section to ensure the client can only submit the name, city, and state. You should not have to restart the server when you're done

```
<fieldset class="form">
  <f:with bean="castle">
    <f:field property="name"/>
    <f:field property="city"/>
    <f:field property="state"/>
  </f:with>
</fieldset>
```

- d. The documentation for the Fields plugin for Grails 3 can be found at <http://grails-fields-plugin.github.io/grails-fields/latest/guide/index.html>

Chapter 12. Using the Google Visualization plugin

1. The Google Visualization Plugin

- a. Check out the plugin documentation at <http://grails.org/plugin/google-visualization>. This plugin shows how to use Google visualizations in your applications without writing any JavaScript.
- b. Open the `build.gradle` file. In the `dependencies` block, add:

```
compile "org.grails.plugins:grails-google-visualization:2.2.1"
```

- c. Open the `GeocoderService` class. Add a method to return the required headers

```
def headers() {  
    [['number', 'Lat'], ['number', 'Lng'], ['string', 'Name']]  
}
```

- d. Add a method to return the latitude / longitude / name triples for each castle

```
def data() {  
    Castle.list().collect { c ->  
        [c.latitude, c.longitude, c.toString()]  
    }  
}
```

2. Using the service in a controller

- a. The `index(Integer max)` action in `CastleController` already returns a list of castles and the total. Add the headers and data to the returned map

```
def index(Integer max) {  
    params.max = Math.min(max ?: 10, 100)  
    respond castleService.list(params),  
        model:[castleCount: castleService.count(),  
            mapColumns: geocoderService.headers(),  
            mapData: geocoderService.data()]  
}
```

3. Updating the view

- a. Open the `index.gsp` file in the `grails-app\views\castle` folder. Add the required library tag to the header.

```
<head>
  ...
  <gvisualization:apiImport />
</head>
```

- b. Somewhere in the body (right above the `<table>` tag is good), add the map tag from the plugin

```
<gvisualization:map elementId="map" columns="{mapColumns}" data="{mapData}" />
```

- c. Below that tag, add a div to hold the map

```
<div id="map" style="width: 100%;"></div>
```

- d. Add a map key for Google Maps that works for localhost

```
google:
  maps:
    key: 'AIzaSyCfjf9I15PFGctk1DUwXK83Vdvng5M71cg'
```

- e. Restart the server and browse to the castle list page. The map should now appear.
- f. To see the castle names when you mouse over them, add the `showTip` property to the map tag

```
<gvisualization:map ... showTip="{true}" />
```

- g. In your browser, do a "view source" to see the generated JavaScript code.

Chapter 13. Switching databases

1. Declare the JDBC driver dependency

- a. Open the `build.gradle` file and add a line that declares the MySQL JDBC driver dependency (the underlying Spring Boot infrastructure supplies a default version number)

```
dependencies {  
    ...  
    runtime 'mysql:mysql-connector-java'  
    ...  
}
```

2. Change the data source settings

- a. Open the file `application.yml` in the `conf` folder
- b. In the `dataSource` block, update settings (other settings can remain as they are now):

```
dataSource:  
  driverClassName: com.mysql.jdbc.Driver  
  username: 'grails'  
  password: 'groovy'
```

- c. In the same block, declare the Hibernate dialect

```
dataSource:  
  // ... as above ...  
  dialect: org.hibernate.dialect.MySql5InnoDBDialect
```

- d. Change the JDBC URL in the development and test environments

```
environments:  
  development:  
    dataSource:  
      dbCreate: create-drop  
      url: jdbc:mysql://localhost:3306/holygrails  
  test:  
    dataSource:  
      dbCreate: update  
      url: jdbc:mysql://localhost:3306/holygrails
```

3. Create the database and user in MySQL

- a. Use whatever client tool you prefer (here a command prompt) to log into MySQL with root access

```
> mysql -u root -p
Password: *****
```

- b. Create the **holygrails** database (don't forget the semicolon)

```
mysql> create database holygrails;
```

- c. Create a user called 'grails' with password 'groovy' for that database only. This assumes you are accessing a MySQL instance on your local machine — adjust the address accordingly if you are connecting to a remote instance

```
mysql> grant all on holygrails.* to grails@localhost identified by 'groovy';
```

4. Access the new database

- a. Use the new database. There shouldn't be any tables yet.

```
mysql> use holygrails;
mysql> show tables;
```

- b. Restart the application. After it starts, there should be 4 tables.

```
mysql> show tables;
+-----+
| Tables_in_holygrails |
+-----+
| castle                |
| knight                |
| quest                |
| task                  |
+-----+
4 rows in set (0.00 sec)
```

- c. Describe a table and check its contents

```
mysql> desc castle;
```

Field	Type	Null	Key	Default	Extra
id	bigint(20)	NO	PRI	NULL	auto_increment
version	bigint(20)	NO		NULL	
city	varchar(255)	NO		NULL	
latitude	double	NO		NULL	
longitude	double	NO		NULL	
name	varchar(255)	NO		NULL	
state	varchar(255)	NO		NULL	

```
7 rows in set (0.02 sec)
```

```
mysql> select * from castle;
```

id	version	city	latitude	longitude	name	state
1	0	Boston	42.3600825	-71.0588801	Camelot	MA
2	0	Natick	42.2775281	-71.3468091	Swamp	MA

```
1 row in set (0.00 sec)
```

Chapter 14. Mapping to existing tables

1. Create the Sakila database



If you already have the Sakila database, skip this step.

- a. Download the zip file for the Sakila database from <http://downloads.mysql.com/docs/sakila-db.zip> and unzip it.
- b. Log into MySQL with root access.
- c. Source the sql files. Supply the full path to the sql files, using forward slashes in the path name.



Detailed installation instructions can be found at <https://dev.mysql.com/doc/sakila/en/sakila-installation.html>

```
mysql> source sakila-schema.sql
...
mysql> source sakila-data.sql
...
```

- d. Verify the number of cities.

```
mysql> use sakila;
mysql> select count(*) from city;
+-----+
| count(*) |
+-----+
|      600 |
+-----+
1 row in set (0.00 sec)
```

2. Create a new Grails app for Sakila

- a. Create a Grails app (in a new directory!)

```
> cd .. // i.e., get outside your 'holygrails' directory
> grails create-app sakila
```

- b. If necessary, add the JDBC driver. In the `build.gradle` file, in the `dependencies` block, add the following line:

```
runtime 'mysql:mysql-connector-java'
```

- c. Change the data source to use the sakila database by editing `application.yml`.

```
dataSource:
  driverClassName: 'com.mysql.jdbc.Driver'
  username: 'root'
  password: ''
  dialect: org.hibernate.dialect.MySql5InnoDBDialect
```

- d. Change the JDBC URL and `create-drop` value in both the development and test environments.

```
environments:
  development:
    dataSource:
      dbCreate: validate
      url: jdbc:mysql://localhost:3306/sakila
  test:
    dataSource:
      dbCreate: validate
      url: jdbc:mysql://localhost:3306/sakila
```

3. Map the Country table

- a. Examine the definition of the COUNTRY table (abbreviated here for width)

```
mysql> desc country;
+-----+-----+-----+-----+-----+
| Field          | Type                | Null | Key | Default          |
+-----+-----+-----+-----+-----+
| country_id     | smallint(5) unsigned | NO   | PRI | NULL             |
| country        | varchar(50)          | NO   |     | NULL             |
| last_update    | timestamp             | NO   |     | CURRENT_TIMESTAMP |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

- b. Create a domain class for the Country table.

```
> grails create-domain-class Country
```



Since no package was specified, this will put the class in the `sakila` package

- c. The default data type for the `id` field is `Long`, but since MySQL is using `SMALLINT` map it to `Short`

```
class Country {
  Short id
}
```

- d. Add a `name` attribute that will map to `country`

```
class Country {
    Short id
    String name
}
```

- e. Add a `lastUpdated` attribute to map to the `last_update` field

```
class Country {
    Short id
    String name
    Date lastUpdated
}
```

- f. Add a `mapping` block to map the attributes to columns

```
static mapping = {
}
```

- g. Map the `id`, `name`, and `lastUpdated` attributes to the corresponding columns.

```
static mapping = {
    id column: 'country_id'
    name column: 'country'
    lastUpdated column: 'last_update'
}
```

- h. Since there's no `version` column, be sure to turn off optimistic locking

```
static mapping = {
    id column: 'country_id'
    name column: 'country'
    lastUpdated column: 'last_update'
    version false
}
```

- i. Add the constraint that the `name` can not be blank

```
static constraints = {
    name blank:false
}
```

- j. Generate a controller and views for **Country**

```
> rails generate-all sakila.Country
```

- k. Start the app and look at the countries through the dynamic scaffolding.

4. Map the City table

- a. Create a domain class for the **CITY** table

```
> rails create-domain-class City
```

- b. Check the definition of the CITY table in the Sakila database

```
mysql> desc city;
```

Field	Type	Null	Key
city_id	smallint(5) unsigned	NO	PRI
city	varchar(50)	NO	
country_id	smallint(5) unsigned	NO	MUL
last_update	timestamp	NO	

```
4 rows in set (0.02 sec)
```

- c. Add attributes to the **City** class

```
class City {  
  Short id  
  String name  
  Date lastUpdated  
}
```

- d. Add the **mapping** block

```
static mapping = {  
  id column: 'city_id'  
  name column: 'city'  
  lastUpdated column: 'last_update'  
  version false  
}
```

- e. Add the constraint that **name** can't be blank

```
static constraints = {  
    name blank:false  
}
```

- f. Add a scaffolded controller for **City**

```
> rails generate-all sakila.City
```

5. Add a **bidirectional, one-to-many relationship with cascade delete**

- a. In **Country**, add a collection of cities

```
static hasMany = [cities: City]
```

- b. In **City**, add the bidirectional association with cascade delete

```
static belongsTo = [country: Country]
```

- c. Add a **toString** method in each class.

```
class City {  
    String toString() { name }  
}  
  
class Country {  
    String toString() { name }  
}
```

- d. (Re)start the server and browse the **City** and **Country** values.

Chapter 15. Calling stored procedures

1. Set up the Sakila data source

- a. If you haven't already done so, set up the Sakila database and data source using steps in the previous exercise.

2. Create a service

- a. Create a service called `StoredProcService` in the `sakila` package. Be careful not to put the word `Service` at the end of the command.

```
> grails create-service StoredProc
```

- b. Replace the existing `serviceMethod` (or whatever the default is) with the stub of our implementation method:

The structure of the `getNumberOfFilmsInStock` method

```
int getNumberOfFilmsInStock(filmId, storeId) {  
    return 0  
}
```

- c. To test a stored procedure, we need to use the real database. Therefore, create an integration test for the new service. The wizard that created the service generated a unit test called `StoredProcServiceTests`, so we need to use a different name.

```
> grails create-integration-test StoredProcServiceIntegration
```

3. Implement the integration test

- a. The documentation for the `film_in_stock` stored procedure is located at https://dev.mysql.com/doc/sakila/en/sakila-structure-procedures-film_in_stock.html. Note that it takes two input parameters, `p_film_id` and `p_store_id`, representing the film and store, and one output parameter, `p_film_count`, which gives the number of copies of that film at that store. The documentation shows that for film 1 at store 1, the procedure returns 4.
- b. Since this is an integration test, we can take advantage of Grails's dependency injection for the service.

```
@Integration  
@Rollback  
class StoredProcServiceIntegrationSpec extends Specification {  
    @Autowired  
    StoredProcService storedProcService  
}
```

- c. Implement the test.

```

@Integration
@Rollback
class StoredProcServiceIntegrationSpec extends Specification {
    @Autowired
    StoredProcService storedProcService

    void 'should be 4 copies of film 1 in store 1'() {
        expect:
        4 == storedProcService.getNumberOfFilmsInStock(1, 1)
    }

    void 'should be 3 copies of film 1 in store 2'() {
        expect:
        3 == storedProcService.getNumberOfFilmsInStock(1, 2)
    }
}

```

- d. Now run the test and watch it fail.



As they saying goes, never trust a test you haven't seen fail.

```
> grails test-app -integration
```

4. Implement the service

- a. Inject the data source into the service

```

class StoredProcService {
    def dataSource

    int getNumberOfFilmsInStock(filmId, storeId) {
        return 0
    }
}

```

- b. Define the stored procedure call using the JDBC escape syntax (inside the `getNumberOfFilmsInStock` method).

```
String sqlTxt = '{call film_in_stock(?,?,?)}'
```

- c. Use the `groovy.sql.Sql` class to connect. Use the injected data source as the argument to the `Sql` constructor.

```
Sql db = new Sql(dataSource) // import groovy.sql.Sql
```

d. Implement the call

```
db.call(sqlTxt, [filmId, storeId, Type.INTEGER]) { count ->
    println count
}
```

e. Since returning from a closure only returns from the closure and not the method, use a temporary variable to return the result. Here is the complete class.

```
class StoredProcService {
    def dataSource

    int getNumberOfFilmsInStock(filmId, storeId) {
        String sqlTxt = '{call film_in_stock(?,?,?)}'
        Sql db = new Sql(dataSource)
        int result = 0
        db.call(sqlTxt, [filmId, storeId, Sql.INTEGER]) { count ->
            println count
            result = count
        }
        return result
    }
}
```

f. The integration test should now run successfully.